第8章 类图和对象图建模

2024/4/21

类模型描述系统内部特征，即类的属性、方法以及类的对象之间的相互关系，它刻画的是系统的静态特征。类模型是面向对象设计中最重要的一种模型。面向对象建模强调围绕对象而不是功能来构建系统，因而类模型用直观的图形化方式表示系统，容易在程序员设计人员之间进行交流和沟通

学习完本章节，要求达到以下状态：

- 能够说明类图和对象图的区别
- 能够说明类图和对象图的表示方法和使用方法
- 能够看懂类图，并理解类和类之间的各种关系
- 能够说明各个类在系统中所承担的不同角色
- 能够理解接口类，抽象类的概念
- 能够借助于对象图来理解类图中类之间的多重度关系
- 能够根据要求绘制类图

8.1 类和对象

- 对象
  - 对象是指具有标识的一个概念、一种抽象或者现实世界一个具体的事物。一些对象在现实世界中有对应的事物，例如，华为技术公司、雷锋、黄继光等，而一些对象是概念，例如二次方程式的求解公式，还有一些对象是为了实现某个功能而引入，没有具体的物理实体，例如4个顶点组成的团以及字符串变量a等
  - 对象是从建模者理解和构造的系统中分离出来的离散单元。它是类的实例，对象具有一个可识别的状态，该状态的在事件触发下可以变化。它是一个其结构和行为都由类来描述的具有身份的个体

8.1 类和对象（续）

- 类
  - 类是有着相同结构、行为和关系的一组对象的描述符号。类中的每一个对象，其属性描述和行为方法都是一样的，但是其状态可能不一样，即其属性的取值不一样，据此，每个对象都可以和同类中的其它对象唯一区分
  - 对象是类的一个具体的实例，而类则是具有相同属相和行为的一组对象的抽象。类是型，对象是值。例如，进程、公司、人、窗口都是类。人这个类中，包含许许多多的人，每个人是一个对象，有自己的身份证号、姓名、年龄、性别等；每个进程是一个对象，有自己的进程号、优先级别、属主等

8.2 类图

- 类图用来描述系统的静态结构，体现类与类之间的关系。类图的元素是应用中有意义的概念，这些概念包括真实世界中的概念、抽象的概念、实现方面的概念和计算机领域的概念，即系统中的各种概念。
- 类图反映了对象的结构。一个面向对象的系统使数据结构和行为特征统一到一个独立的对象结构中。
- 类图中的关键元素是类及它们之间的关系。类图描述系统内部对象的特征、对象之间的相互关系以及对象所属的每个类的属性和操作，捕获系统的静态特征。类之间的关系有关联、泛化及各种不同的依赖关系，包括实现和使用关系。

8.2 类图

◆类图，就是用于对系统中的各种概念进行建模，并描绘出它们之间关系的图。类图展示了系统的逻辑结构。

◆在大多数的 UML 模型中，我们可以将这些概念的类型概括为以下四种，分别是：

1 类　　2 接口　　3 数据类型　　4 构件

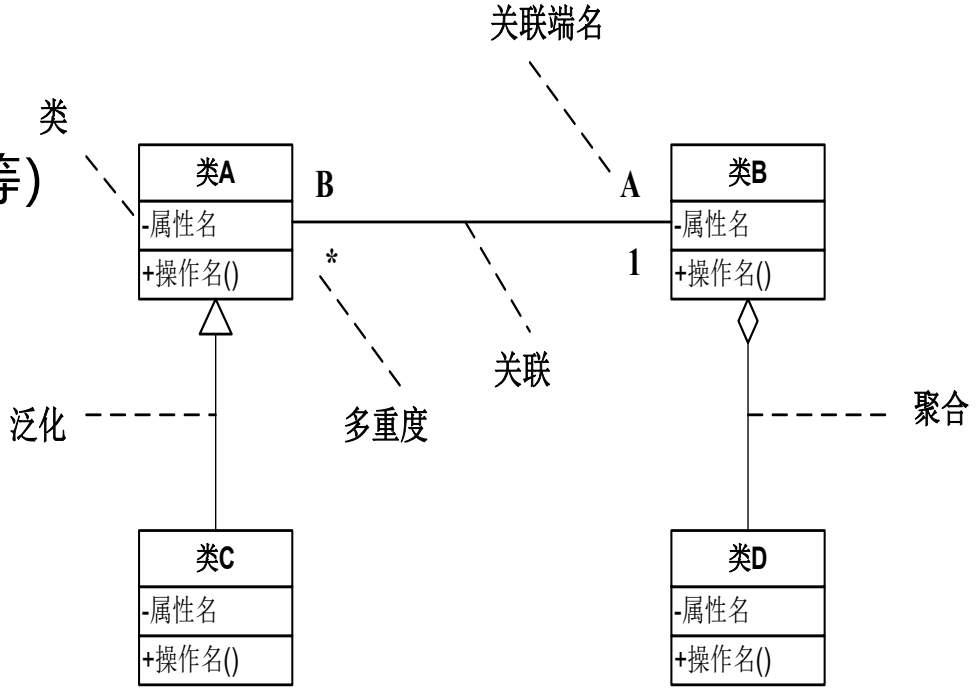
◆ UML给这些概念的类型定义了一个特别的名字：类元(Classifer)

--类元是比类更为通用的术语。参与者，信号，节点,组件等也是类元。

--类元是描述一组具有相同特征的UML元素的抽象，并且这种抽象具有 在 UML（统一建模语言）中，Classifier（类元）是表示系统内一个集合的元素的抽象概念。它们包含了一组共同的功能或结构特征。

8.2 类图-类图概要

- 类是着眼于现实中存在的物体（对象）的属性，行为的共同特性，并进行抽象后得到的。
- 类图用来描述系统的静态结构和系统中类之间的静态关系。
- 一个系统中，可以从不同侧面做出多个不同的类图。一个类图是描述系统静态结构的某个侧面的视图（view）。
- 类图由以下元素组成:
  - 类（类名，属性，操作）
  - 关系（关联，泛化，聚合等）

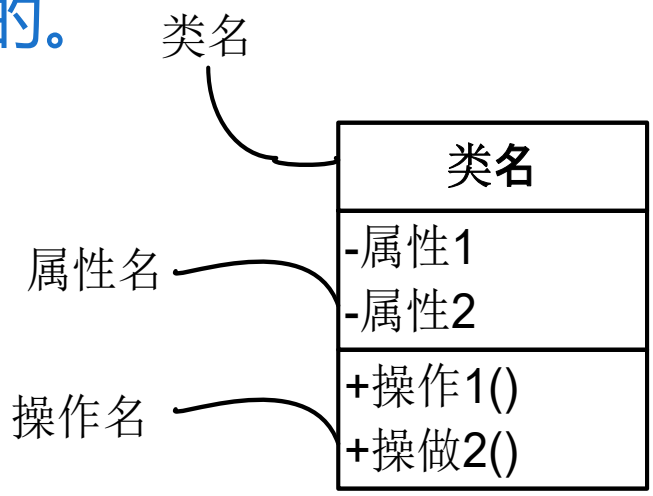


8.2.1 类建模

- 类建模代表了应用领域中的离散概念，例如：物理实体（如飞机）、商业事物（如一份订单）、逻辑事物（如广播传输）、应用事物（如取消键）、计算机领域的事物（如哈希表）或行为事物（如一项任务）。所有的属性与操作都被附在类或其他类元上。类建模是面向对象系统建模的核心。

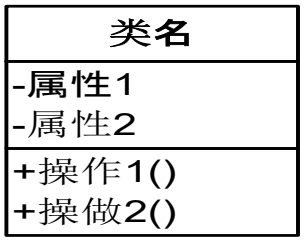
8.2.1类建模-类的表示方法

- 类是用划分为3个区域的矩形框来表示的。
  - 上面的区域：类名
  - 中间的区域：属性
    每行一个，该区域可省略。省略并不代表没有。
  - 下面的区域：操作
    每行一个，该区域可省略。省略并不代表没有。

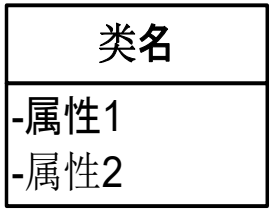


- 表示类有以下几种方法：

●显示属性和操作的区域



●不显示操作的区域



●不显示属性的区域



●不显示属性和操作的区域



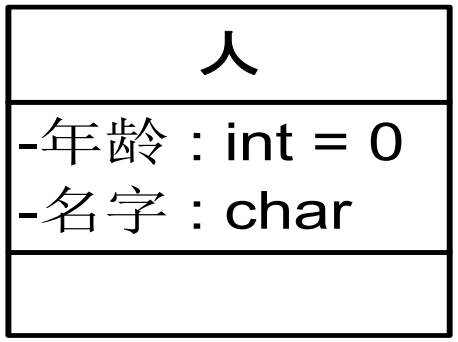
注：也可通过【包名：类名】方式来表示类名，明确该类属于哪个包。

8.2.1类建模-类的属性

- 在构成类的信息中，将知识性的静态信息称为属性。
  - 类生成的所有对象都拥有类中定义的属性。
  - 每个对象拥有独自的属性值。

- 属性的表示方式如下：

属性名： 类型表示 = 初始值





-按照标准的约定和最佳实践，属性名称通常是以小写字母开始的，采用驼峰命名法（CamelCase）。这个命名约定是为了区分类名（Classes），类名通常以大写字母开始。
-但这并不是UML规范的强制要求。

注：属性类型和初始值也可省略

8.2.1类建模-派生属性

- 派生属性是通过其他属性计算得出的属性称为派生属性。
- UML表示派生属性时，在属性名前加“/”来标识。派生属性是一种冗余的属性，虽然没有独立存在的必要性，但是在计算派生属性之值比较花时间时，它可以提高程序的性能和效率。

下面的列子中，类模型包含一个属性“出生年月”，同时还包含一个派生属性“/年龄”。当软件程序运行中，需要使用年龄时，就不需要通过“年龄=当前的日期-出生年月”这样一个表达式来计算，因此，年龄属性就是一个可以不声明的属性，是一个派生属性，它出现的目的是减少程序运行时的一些运算过程

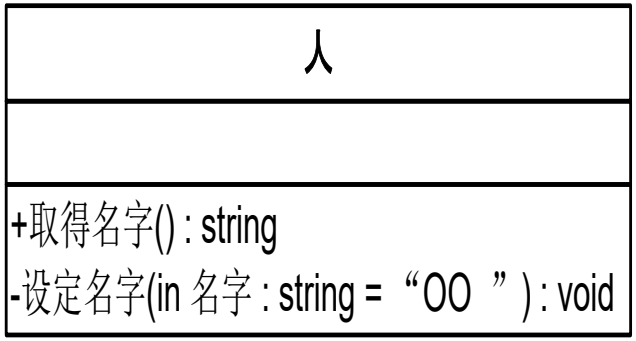


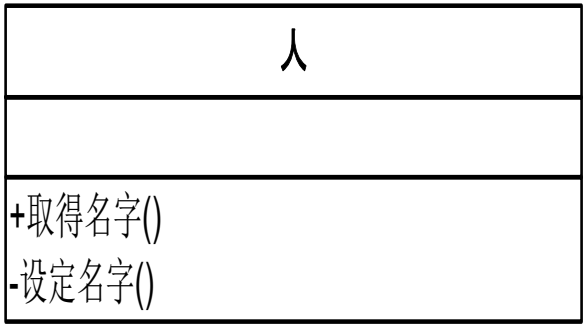
8.2.1类建模-类的操作

- 在构成类的信息中，将行为（动态）信息称为操作。
  - 类生成的所有对象中都拥有类中定义的操作。
  - 可以针对该类的对象进行操作的调用。
- 操作的表示方式如下：

[可见性] 操作名（输入输出类别 参数名：类型 = 默认值）：返回值类型

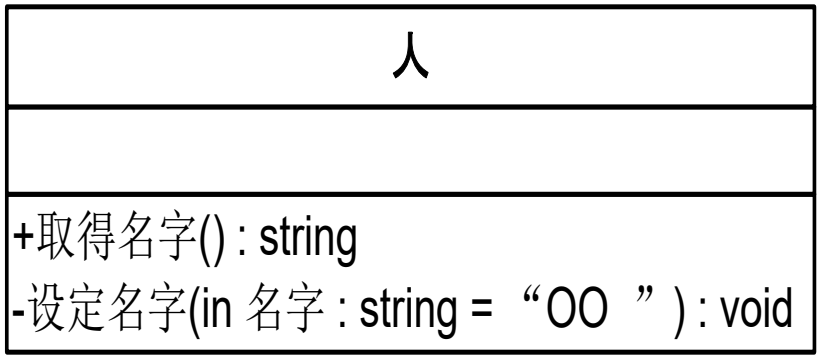
  - 参数的输入输出类别，可以指定以下中的任何一种：
    - in　输入；省略时的默认值
    - out　输出
    - inout　输入输出
  - 多个参数的情况下，用逗号（，）隔开
  - 输入输出类别，参数名，参数类型，默认值，返回值类型均可省略。





8.2.1类建模-可视性

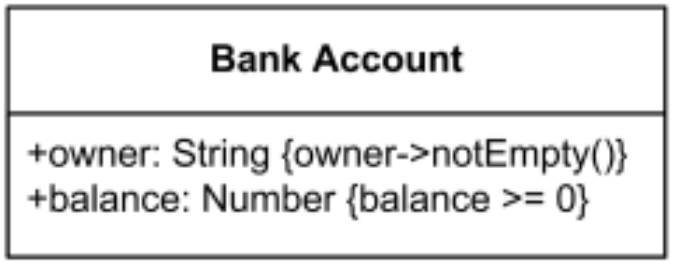
- 一个类的属性或者操作，能不能被其他类进行访问，称为可视性。在类图中可为类的属性和操作指定其可视性。
- UML规定了4种可视性：
  - 【+】（public）……所有的类都可以访问
  - 【-】（private） ……只有自己可以访问
  - 【#】（protected） ……只有自己及其子类可以访问
  - 【~】（package） ……只有自己和同一包内的类可以访问



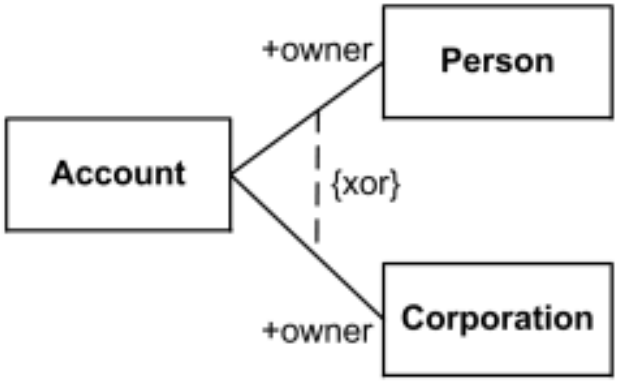
上述类中：操作【取得名字】是其他所有类都能访问的。
操作【设定名字】是只有自己可以访问调用的。

约束

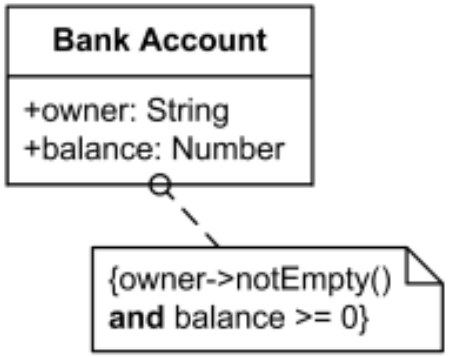
- 类图可以使用约束，表示类或者关系上的条件、限制或者断言。包括：
  - 预期的属性组合
  - 类实例的有效状态
  - 方法如何能够被调用
  - 方法调用的先后顺序
  - 特定属性的值域
  - 对象之间特定关联或依赖的存在性



*Bank account **attribute constraints***
- non empty owner and positive balance.



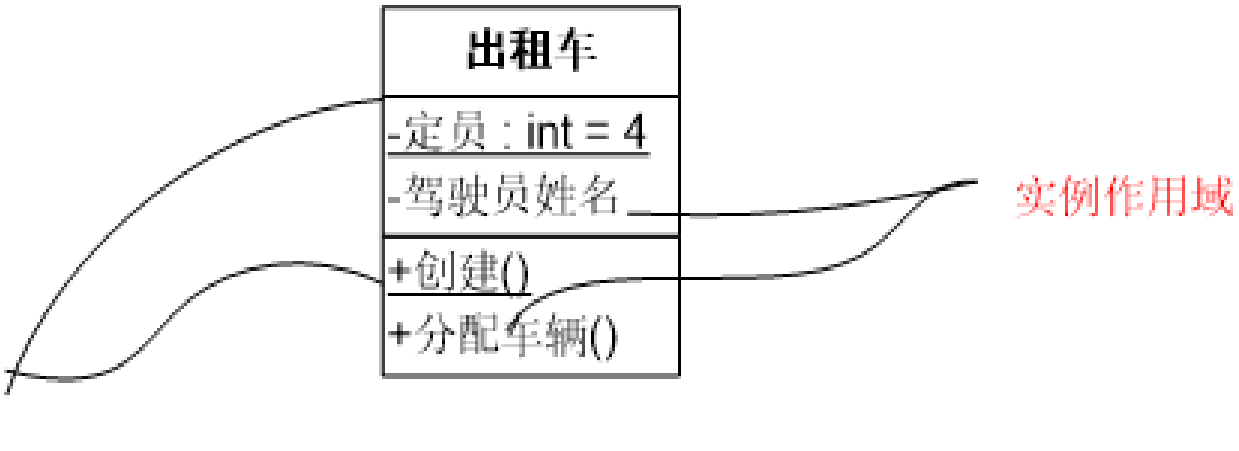
Account owner is either Person or Corporation,
***{xor}** is predefined UML constraint.*



*Bank account **constraints** - non empty owner and positive balance.*

8.2.1类建模-作用域

- 类作用域
  - 类的所有对象能共享的属性值或者操作，称为类作用域属性或者类作用域操作。
  - 在属性名或者操作名下画一条下划线，或者前面加上{static}以示标识。
  - 类作用域的属性值在该类的所有对象中共享。
  - 类作用域的操作是通过调用该类的操作来实现的。
- 实例作用域
  - 类的所有对象拥有各自独立的值的属性称为实例作用域属性；需要通过访问对象来调用的操作称为实例作用域操作。
  - 默认情况下的属性和操作都是实例作用域的。
  - 实例作用域的操作要通过调用具体对象的操作来实现的。



8.2.1类建模-作用域的例子（属性）

出租车
-定员 : int = 4 -驾驶员姓名
+创建() +分配车辆()



驾驶员姓名 = 张三

驾驶员姓名 = 李四

驾驶员姓名 = 王五

每个实例都拥有各自的属性值

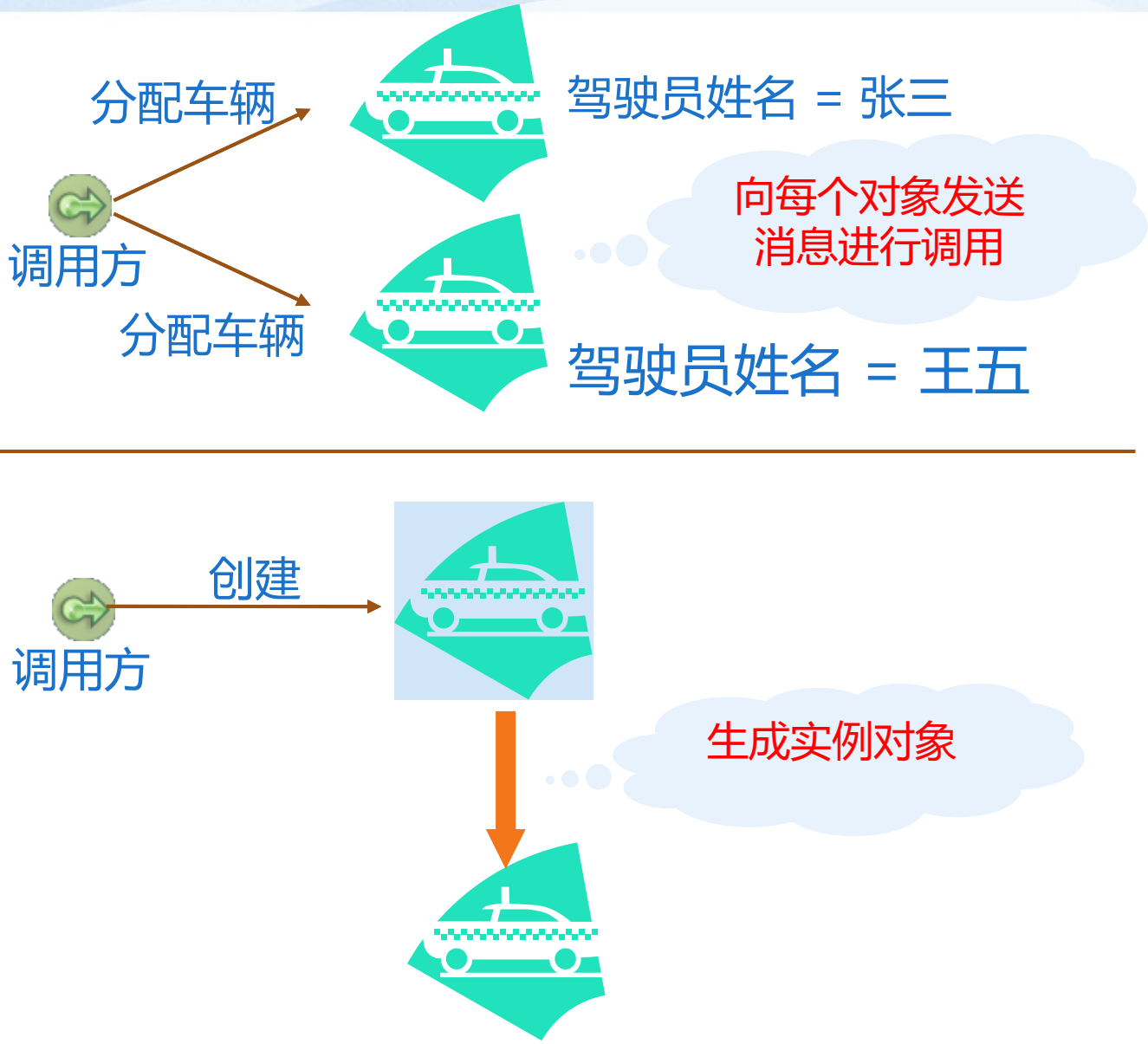


定员 = 4

类的全部实例共有该属性值

8.2.1类建模-作用域的例子（操作）





8.2.2类建模-类之间的关系

- 类除了具有属性和操作之外，类之间还具有一定的关系。

  - 系统行为是通过类的实例对象之间的协调动作来是实现的。为了实现对象之间的协调动作，类之间就需要一定的关系。

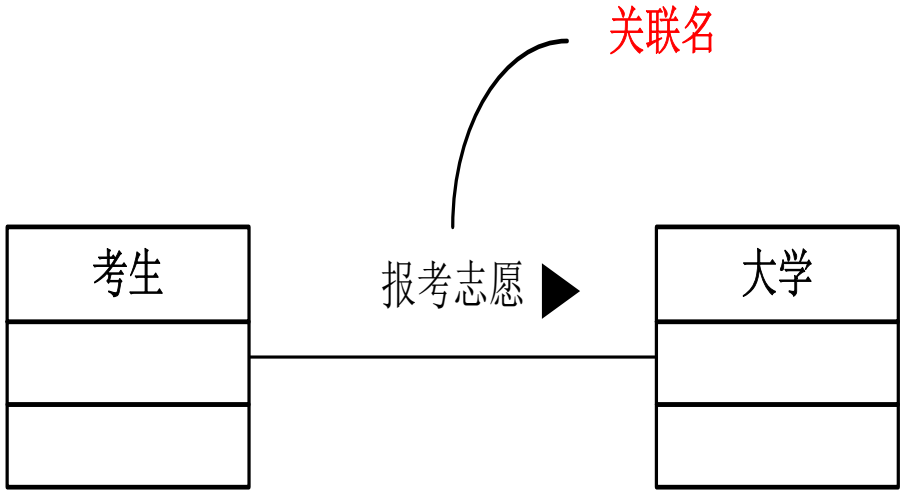
- 正确表示类之间的关系，才能正确表示系统的结构模型 。
- 类之间常见的关系有以下几种
  - 关联
  - 依赖 （实现）
  - 聚合，组合
  - 泛化

关联（association）

- 关联表示的是类之间的一种相对永久的结构性关系。
- 用实线来连接两个有关联关系的类 。实线两端的类称为关联端。
- 类之间的关联还包含如下要素：
  - 关联名
  - 关联端名
  - 多重度
  - 关联方向

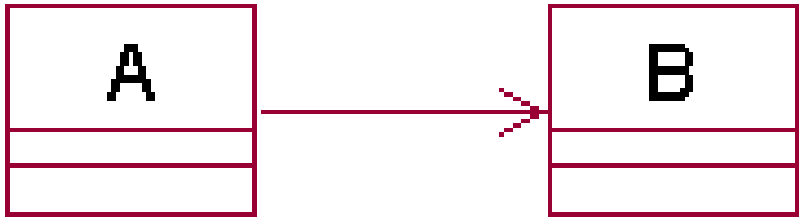
关联名(association name)

- 为了明确关联的意义时，可以给关联起一名字。也可省略。
- 关联名一般都是动词。
- 关联名具有方向，一般是在关联名后跟黑三角来表示关联名的方向。



“考生”类和“大学”类是通过“报考志愿”关联起来的。而且“报考志愿”关联是由“考生”发起“大学”接受的关联。

◆例：单向关联所生成的Java代码：(可以选择是否生成类的缺省构造方法)




```
public class A
{
  public B theB;

  /**
   * @roseuid 3DAFBF0F01FC
   */
  public A()
  {

  }
}
```

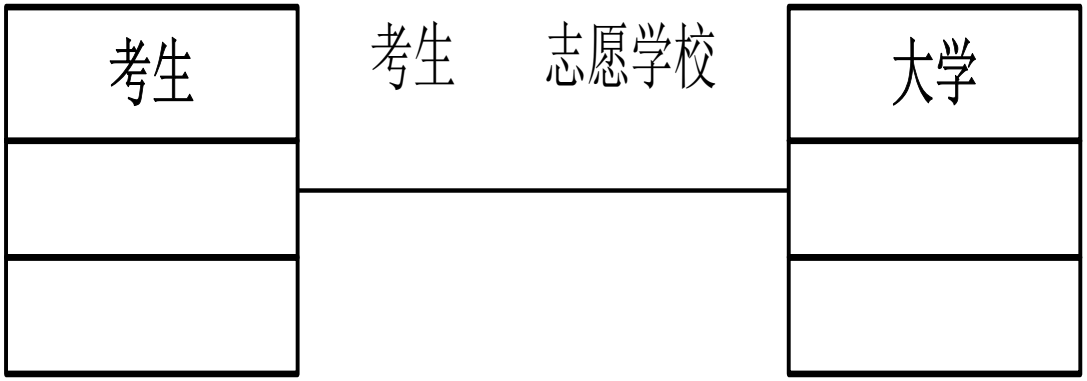
```
public class B
{

  /**
   * @roseuid 3DAFBF0F01A2
   */
  public B()
  {

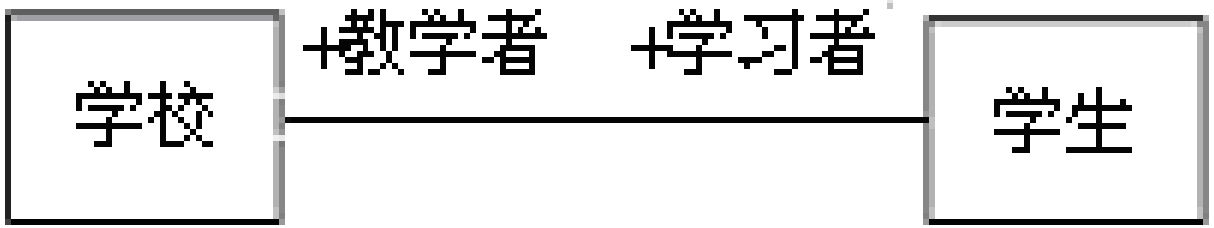
  }
}
```

关联端名(association end name)

- 表示关联的一端相对于另一端的特征，功能和立场。
- 写在关联端处。



- 关联端名称通常称为角色名称(role name)（但在 UML 2.4 标准中并未这样定义）。角色名可以用可见性修饰。

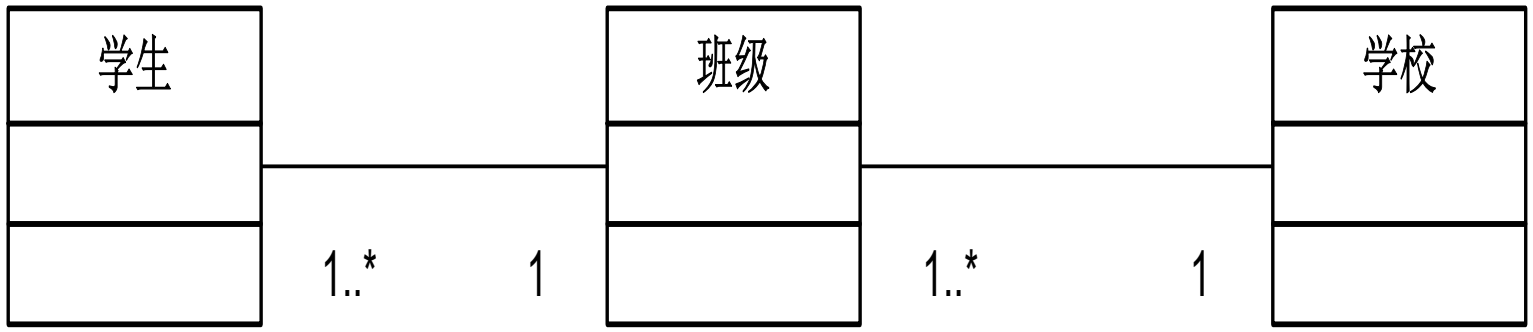


多重度

- 表示相关联的类的实例对象之间的对应关系。
  - 关联的一端的类的一个实例对象，与之有关系的另外一端的类的实例对象的数目

- 多重度写在关联端处，常见的几种情况
  - 必须是1　1
  - 0或者1　0. .1
  - 0以上　0. . * 或 *
  - 1以上　1 . . *
  - 指定区间　1 . . 10

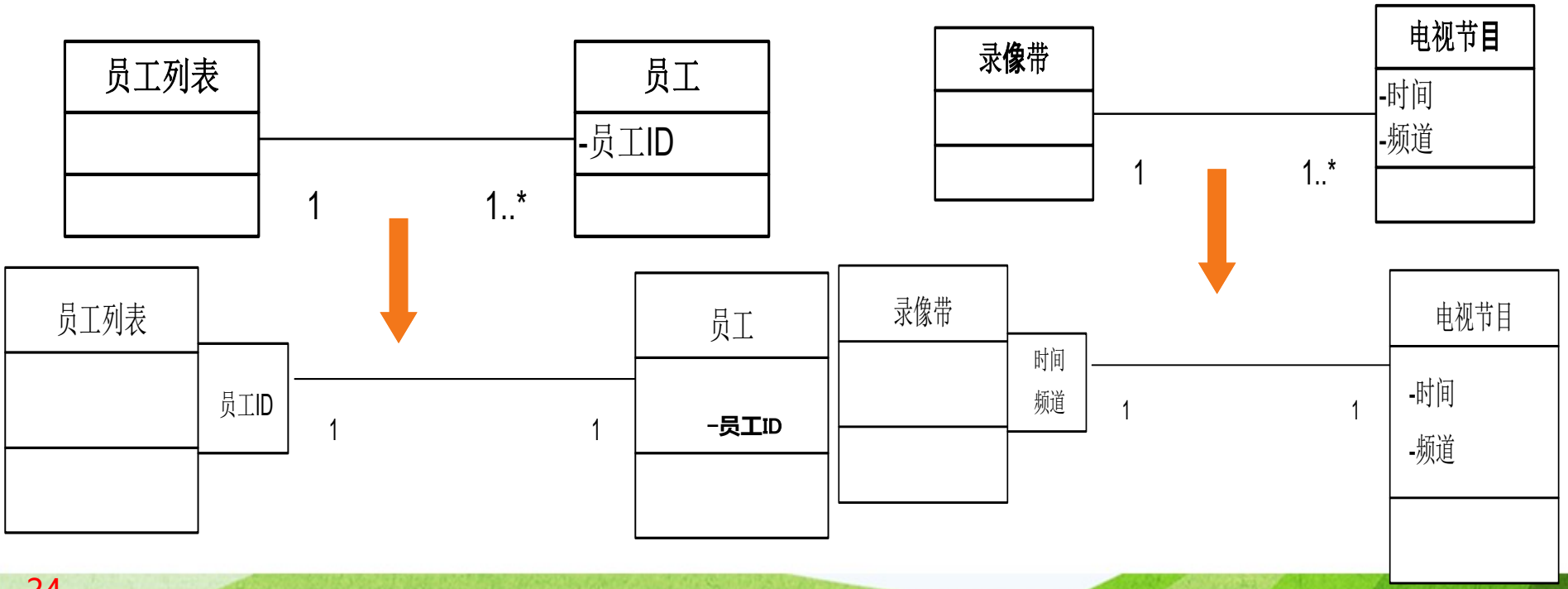
修饰符	语义	修饰符	语义
0	仅为0个	n，*，0..*	任意多个
0..1	0个或1个	3	3个
0..n	任意多个	1..n,1..*	1到任意多个



限定符

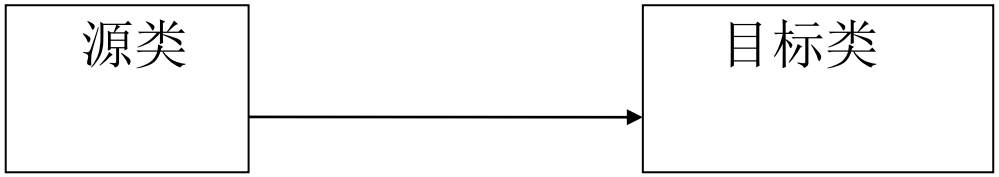
- 根据类的属性，为了减少相关联的对象数目而设置的条件
  - 可减少类之间关联的多重度。
  - 限定符可以理解为一种键（key）或索引，它将关联端的一个集合缩减为更小的、更具体的集合或单个元素。在一定程度上，限定符类似于数据库领域中的键字段，它用于加强数据检索的效率和准确性。

- 限定符用小矩形框表示，矩形框加在关联端上，矩形框内要写明限定符属性。
  - 限定符属性对应于关联的另一侧类中的一个属性。
  - 当限定符有多个时，在限定符框里分行表示。



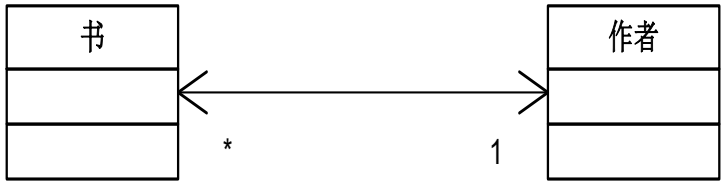
关联方向(导航性，Navigability)

◆导航性描述了以源类创建的对象可以将消息发送给由目标类创建的对象，反之，不能以目标类创建的对象的消息发送给源类创建的对象。导航性表示箭头从源类指向目标类。即，消息可以从源类对象发送到目标类对象，反之，不可以。



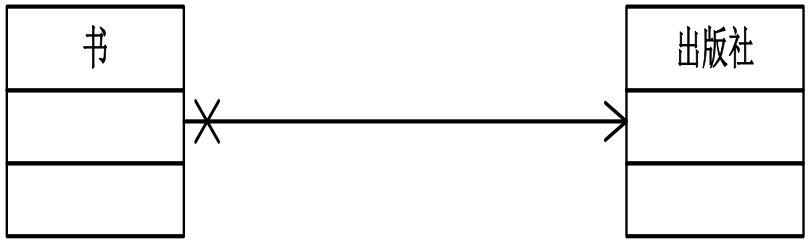
关联方向(导航性，Navigability)

- 在设计阶段，可以根据需要明确关联的方向性
  - 关联的两个类互相之间都会去向对方发送消息的情况下，称为双向关联。关联端两端都要用箭头明示。



  - 关联的两个类之间，只能由一端向另外一端的类发消息，而反过来不行时，称为单向关联。

接受消息一方的关联端用箭头表示，不能接受消息一方的关联端用“X”进行表示。

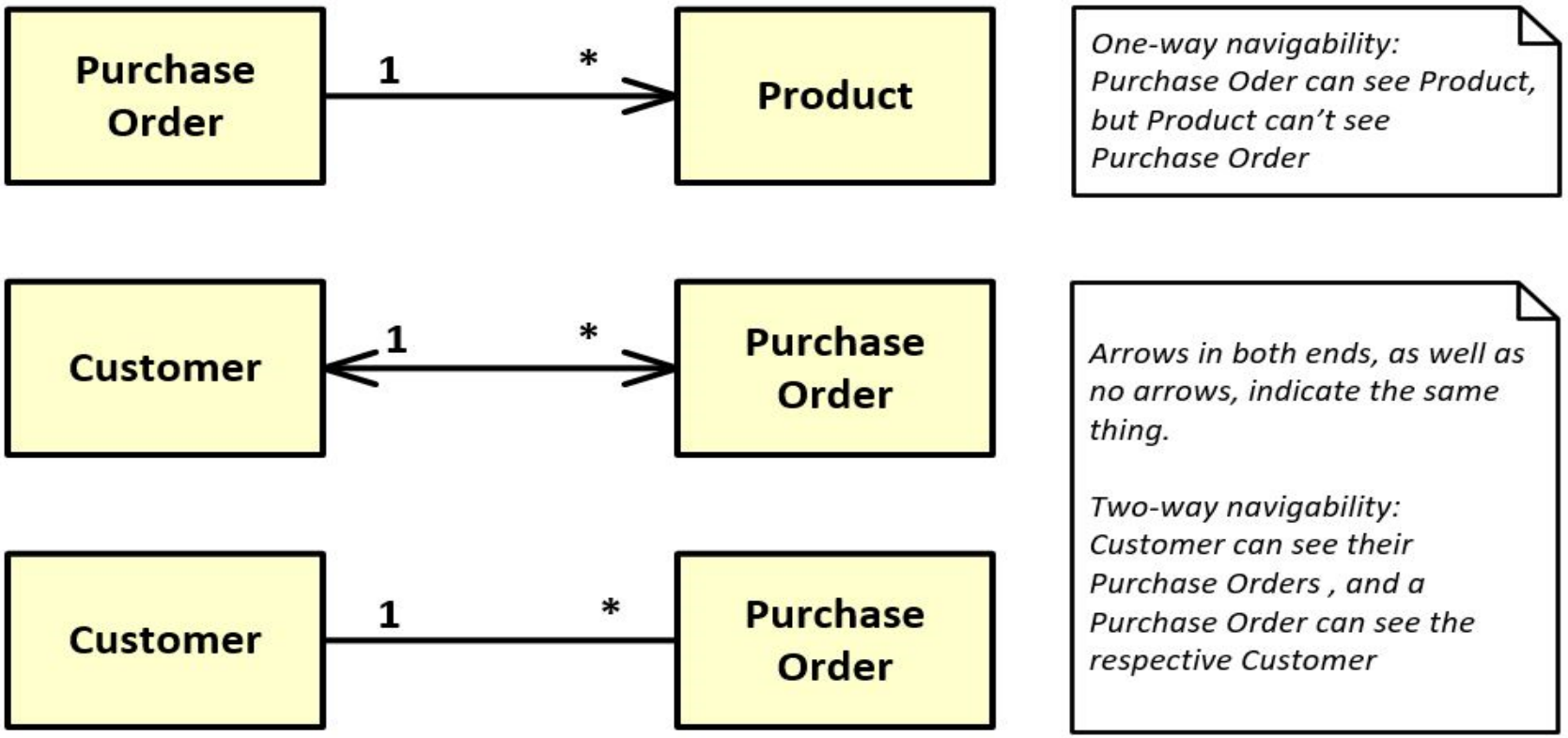


  - 关联的某一端的关联方向尚未明确的情况下，可不标明方向性。（在UML1.x中，不明示关联方向的情况等同于UML2.0中的单向关联“X”）



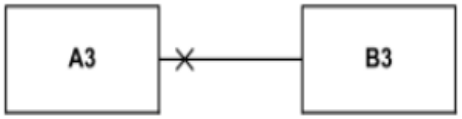
关联方向(导航性，Navigability)

- 双向箭头或者双向无箭头均可表示“双向可导航".无箭头有时表示”未规定"

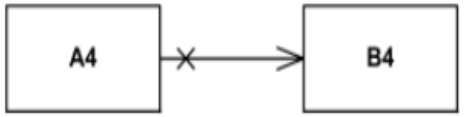


关联方向(导航性，Navigability)

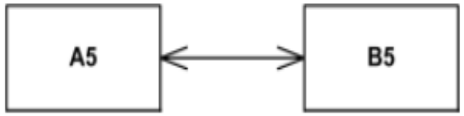
◆UML 2.4规定了"X"代表不可导航；“●”代表拥有该关联的类元。(了解即可.)



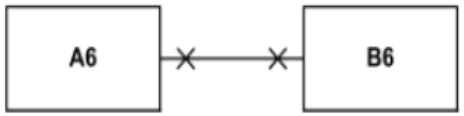
*A3 is **not navigable** from B3 while B3 has **unspecified navigability**.*



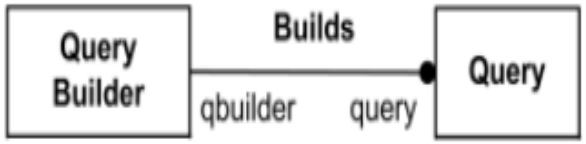
*A4 is **not navigable** from B4 while B4 is **navigable** from A4.*



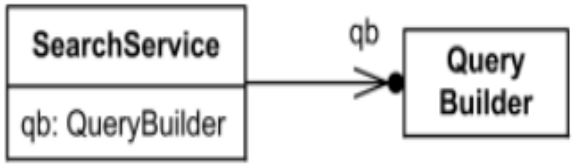
*A5 is **navigable** from B5 and B5 is **navigable** from A5.*



*A6 is **not navigable** from B6 and B6 is **not navigable** from A6.*



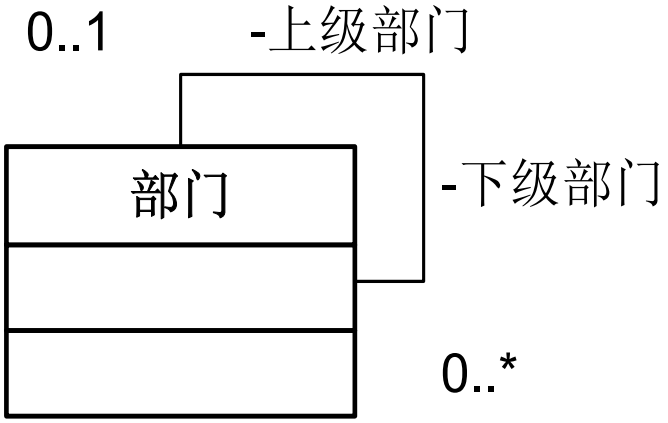
*Association end **query** is owned by classifier **QueryBuilder** and association end **qbuilder** is owned by association **Builds** itself*

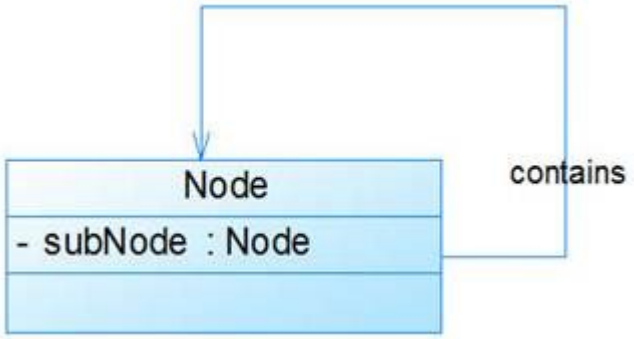


***Association end** qb is an **attribute** of SearchService class and is **owned** by the class.*

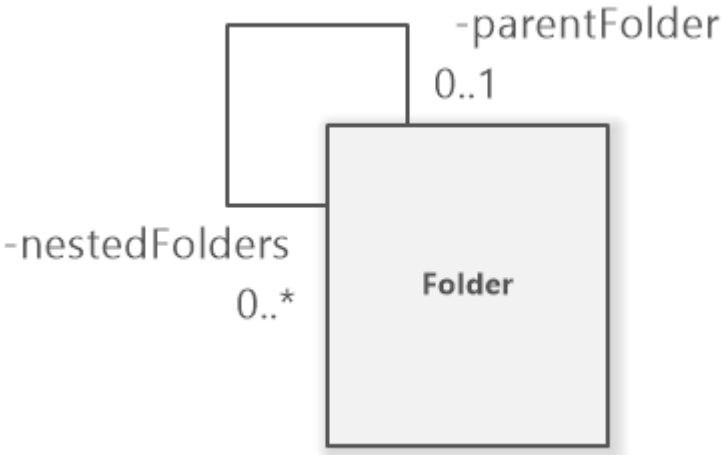
递归关联

- 一个类和自身具有某种关联时，称为递归关联。也称为自关联或自引用关联。
  - 同一个类的多个对象之间具有某种关系时，称为递归关联。同一个类的同一个对象之间具有某种关系时，不称为递归关联。



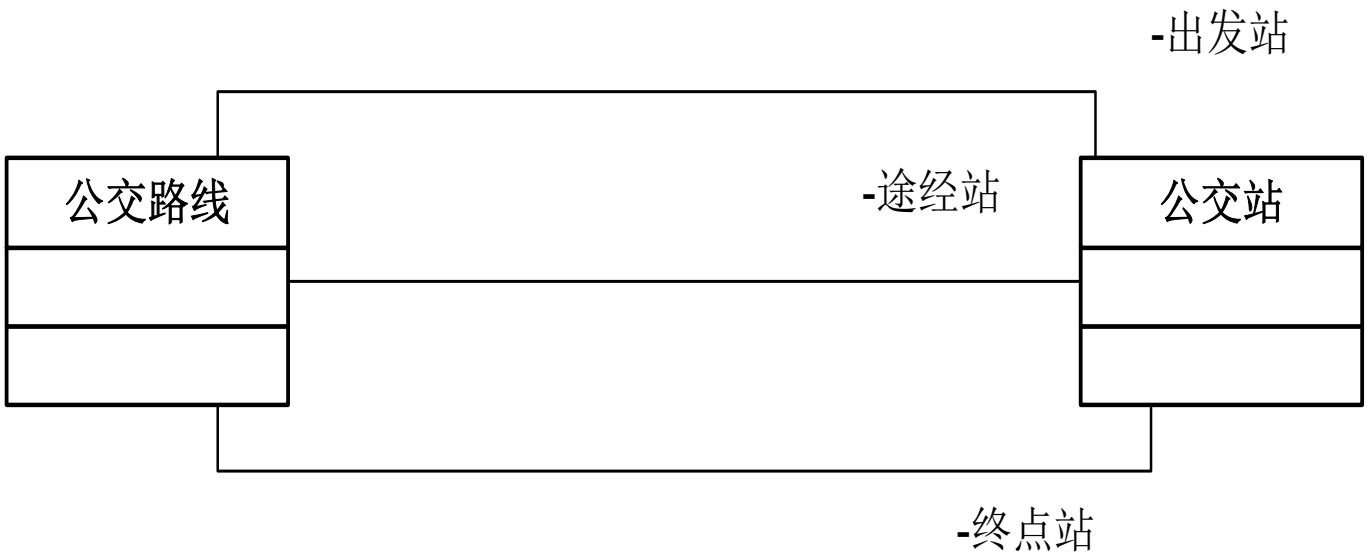


一个部门拥有0个或多个下属部门。
一个部门拥有0个或1个上级部门。

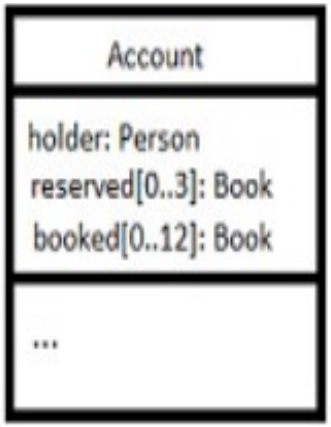


多重关联

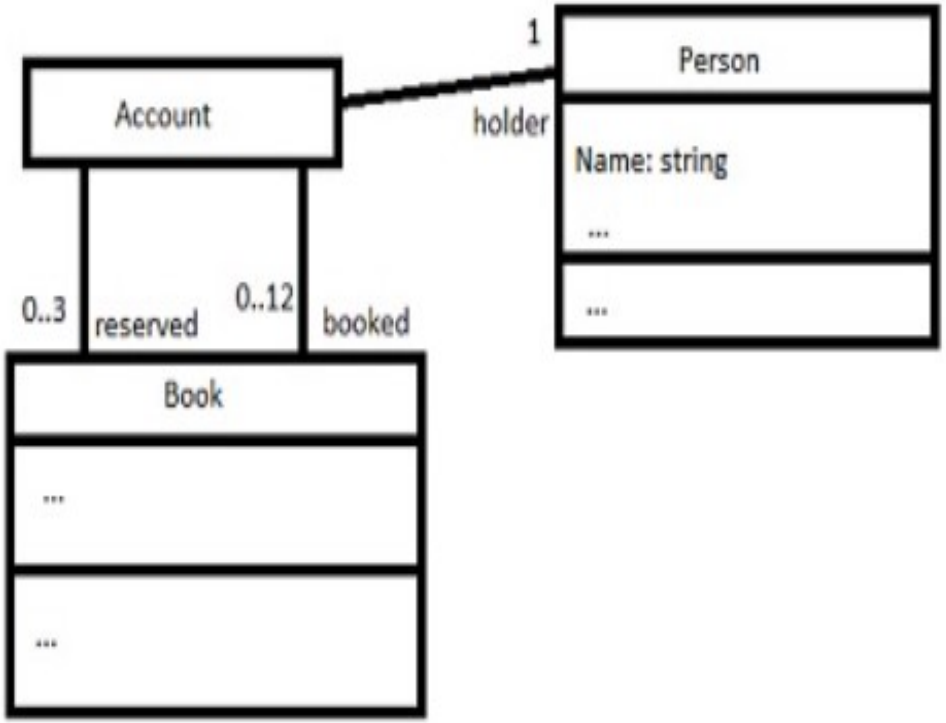
- 两个类之间具有多个不同意义的关联。
  - 每个关联都画一条关联线。
  - 可以通过附加关联名或者关联端名来进行区分。



多重关联

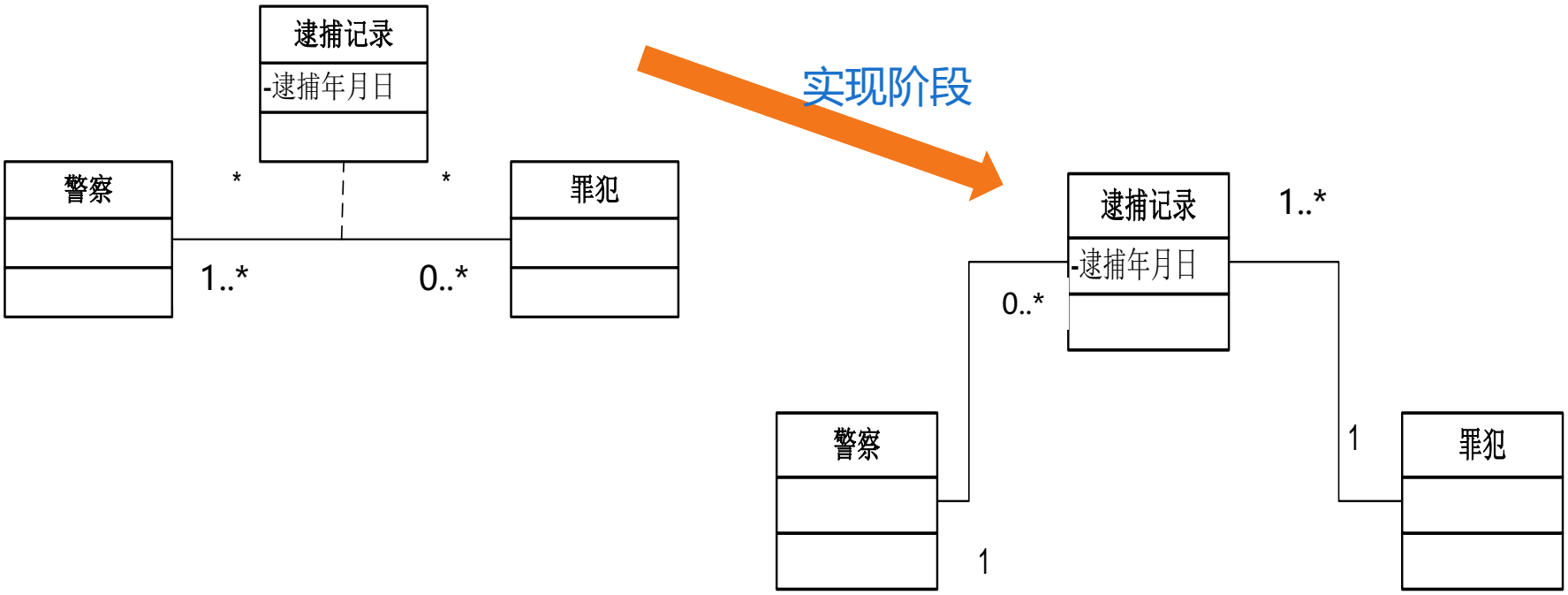


is the same as



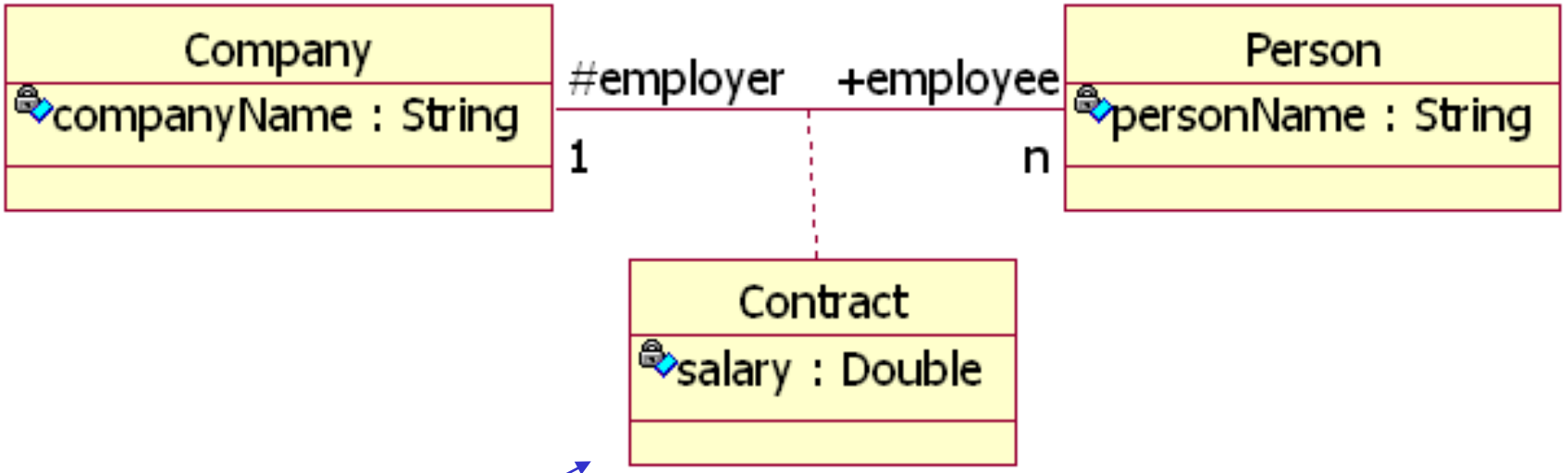
关联类

- 拥有类之间的关联信息的类称为关联类。
  - 类之间的关联多重度为“多：多”的时候，为了方便管理关联信息而使用。
  - 在代码实现阶段，要把概念意义上的关联类转化为实体类来进行实现。(引入实体类，转换为二元关联)
  - 同普通类一样，可定义属性，操作和关系。
  - 当相关联的两个类的对象一旦确定，该关联类的对象也能唯一确定。
  - 关联类和关联线之间用虚线相连接。。



◆关联本身也有特性，通过**关联类(association class)**可以进一步描述关联的属性、操作，以及其它信息。关联类通过一条虚线与关联连接。

◆例：关联类的例子。

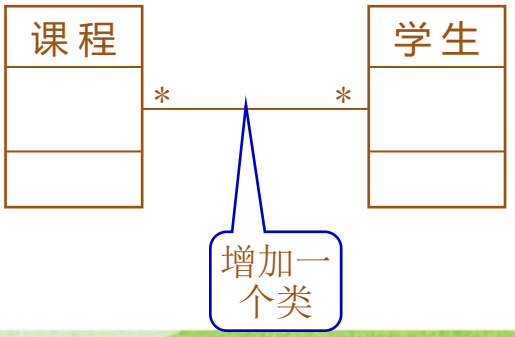


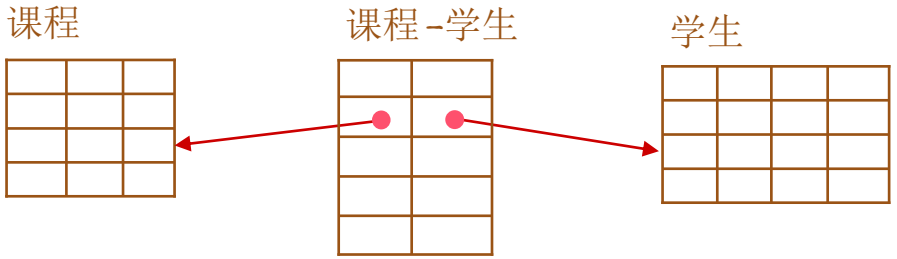
二元关联的实现（多对多）

问题：任何一端的一个对象实例的要和另一端多个对象实例发生关联，而且数量不确定。实现时不知道该设立多少个指针（或者对象标识、外键）才能够用。

编程语言：用两个类分别实现关联两端的类，同时用另外一个类来实现它们之间的关联。实现关联的类含有两个属性，分别是指向两端的类的对象实例的指针或者对象标识。

关系数据库：用两个数据库表分别实现关联两端的类，同时用另外一个数据库表来实现它们之间的关联。实现关联的数据库表含有两个属性，它们分别是指向两端的表的元组的外键。

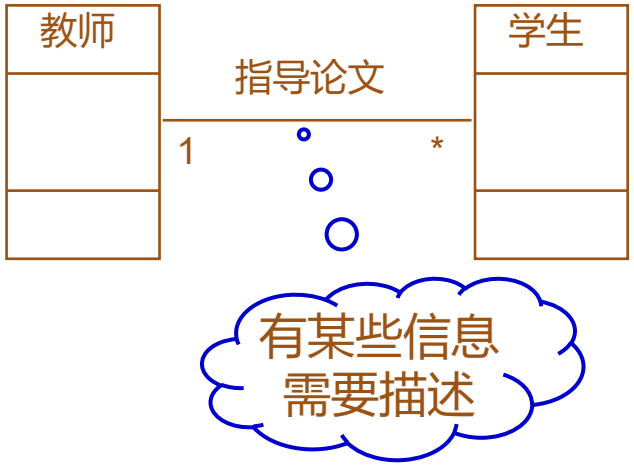


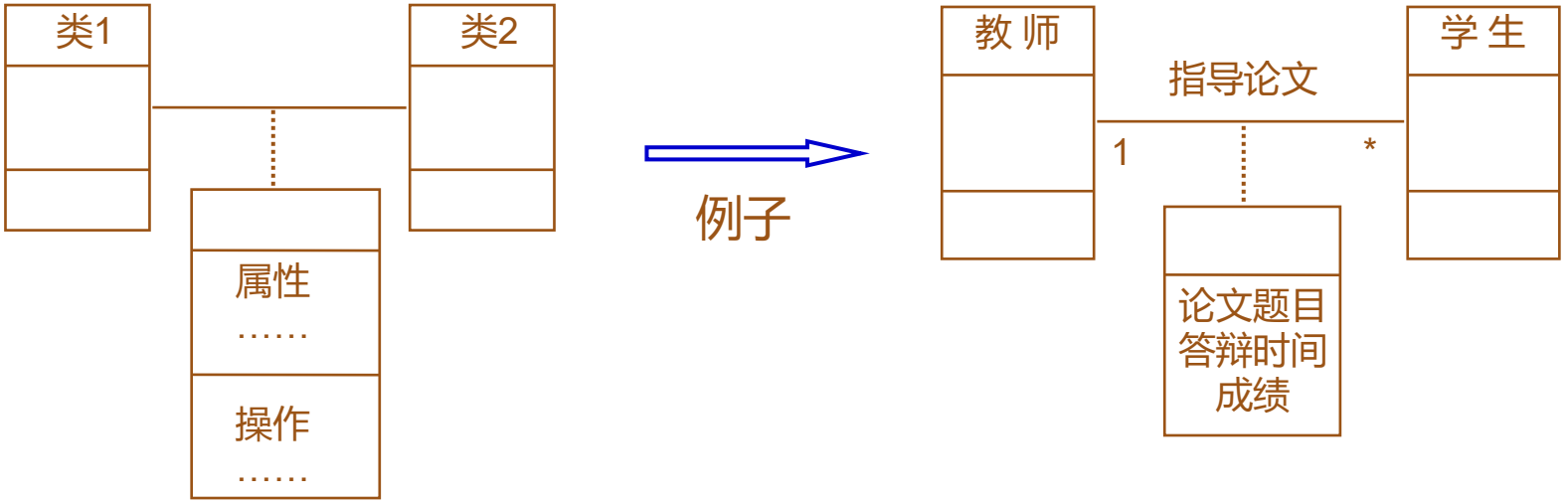


运用简单的概念及表示法解决各种复杂的关联问题

（1）带有属性和操作的关联

OMT（及UML）的概念扩充关联类（association class）





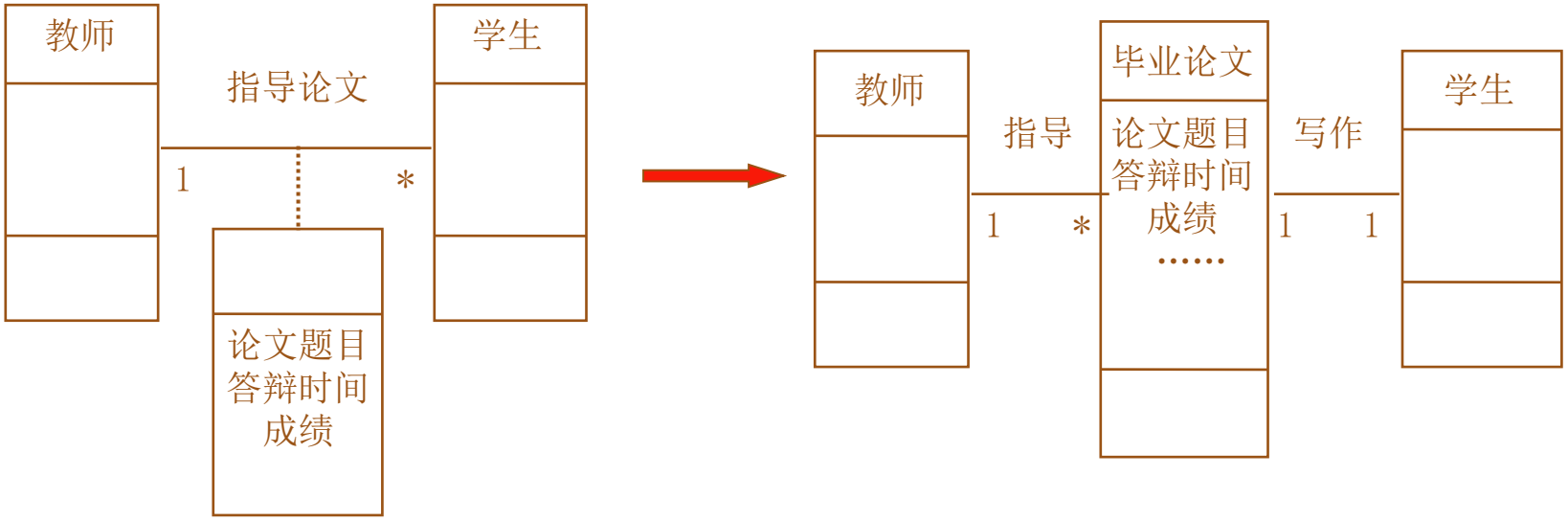
问题：增加了概念的复杂性，缺乏编程语言支持

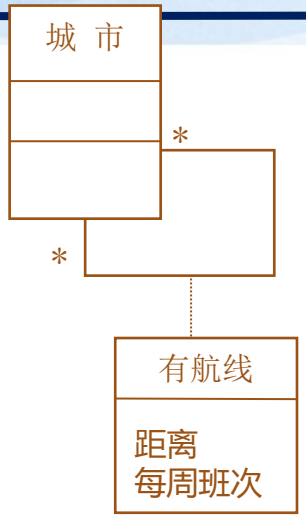
换一种思路考虑问题：

两类对象之间的关联带有某些复杂的信息，说明它们之间存在着某种事物（尽管可能是抽象事物）。

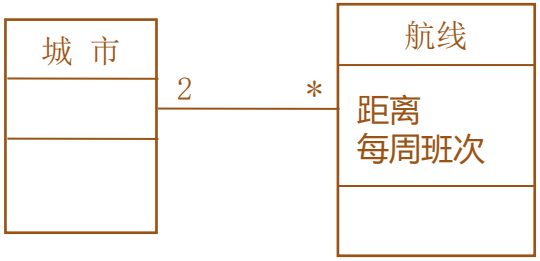
用普通的对象概念来表示这种事物，简化关联，减少概念，并加强与OOPL的对应。

例1

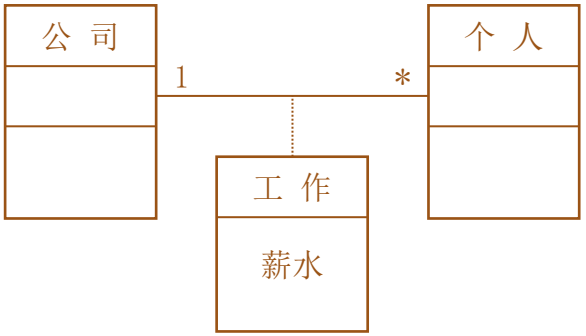




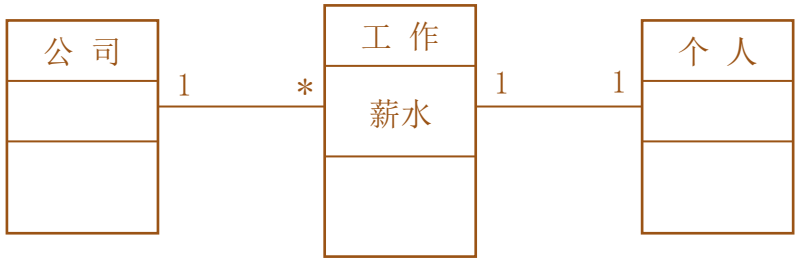
城市之间有航线



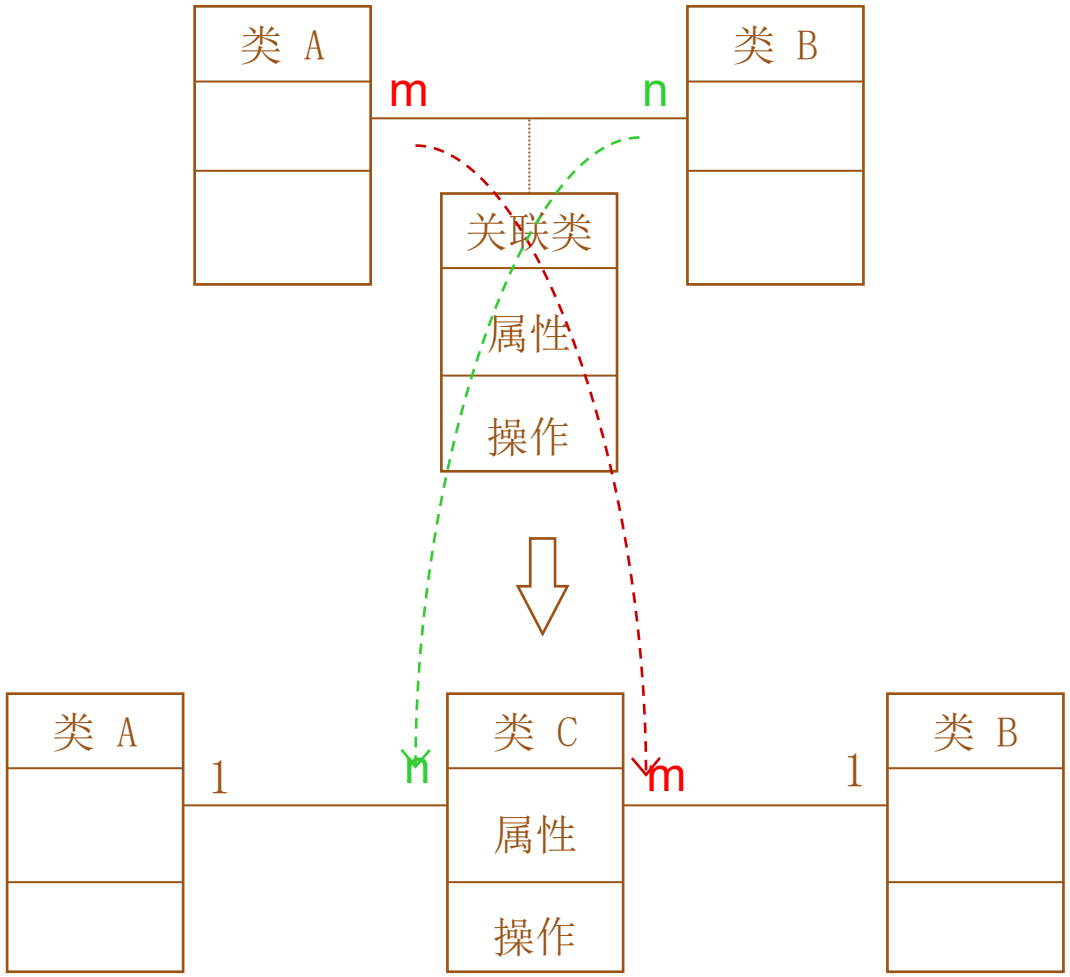
城市之间存在航线对象



公司与个人



公司与个人之间存在工作对象



把通过关联类表打的复杂关联转化为简单的二元关联，把关联类定义为普通的类，切断原来的关联连接线。利用新增加的类分别建立它们之间的关联。

注意在转化中多重度的变化规律：转化之后A和B的数量约束均为1，而m和n互换位置。

◆所生成Java代码示例：（说明：在设置代码生成属性时，设置为不生成类的缺省构造方法）

```
//Source file: F:\\code\\Company.java

public class Company
{
   private String companyName;
   public Person employee[];
}
```

```
//Source file: F:\\code\\Contract.java

public class Contract
{
   private Double salary;
}
```

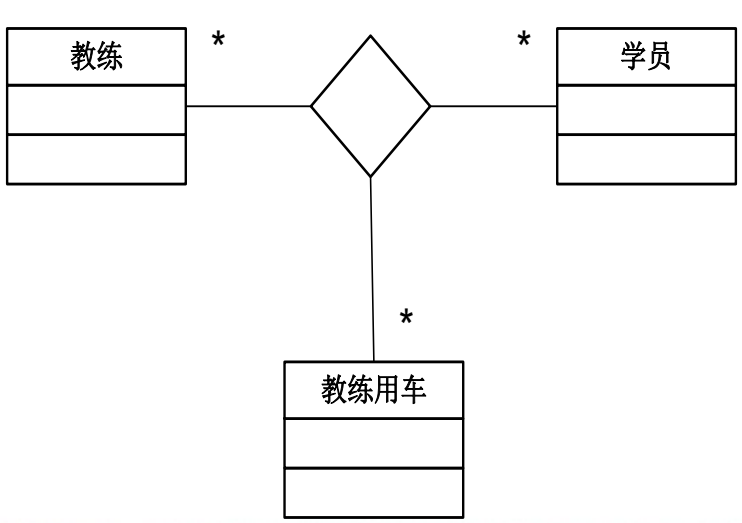
```
//Source file: F:\\code\\Person.java

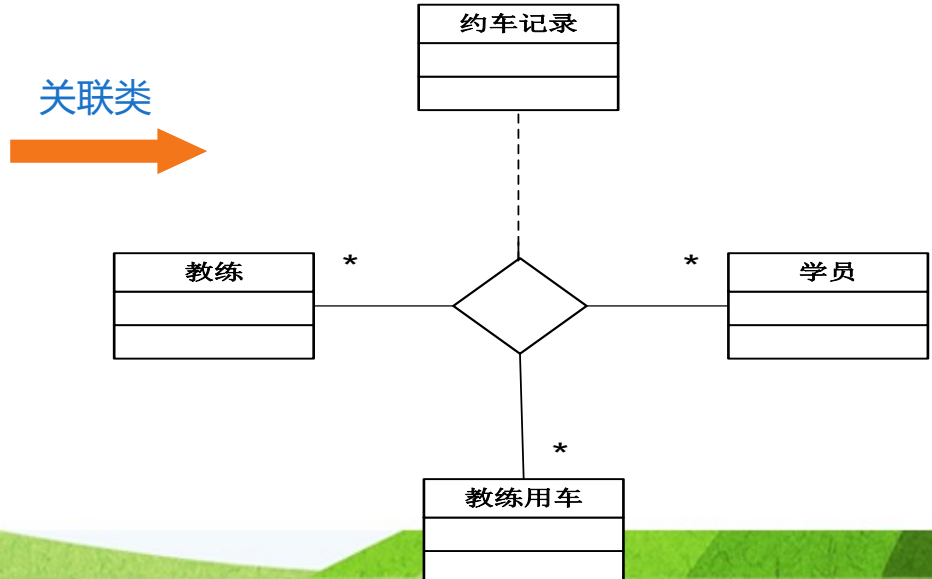
public class Person
{
   private String personName;
   protected Company employer;
}
```

N元关联及其关联类

- 三个及三个以上的多个类之间互相存在的关联称为N元关联。
  - N元关联用菱形框表示，各个相关联的类和菱形框之间用实线项链。
  - N元关联不能使用聚合和限定符。

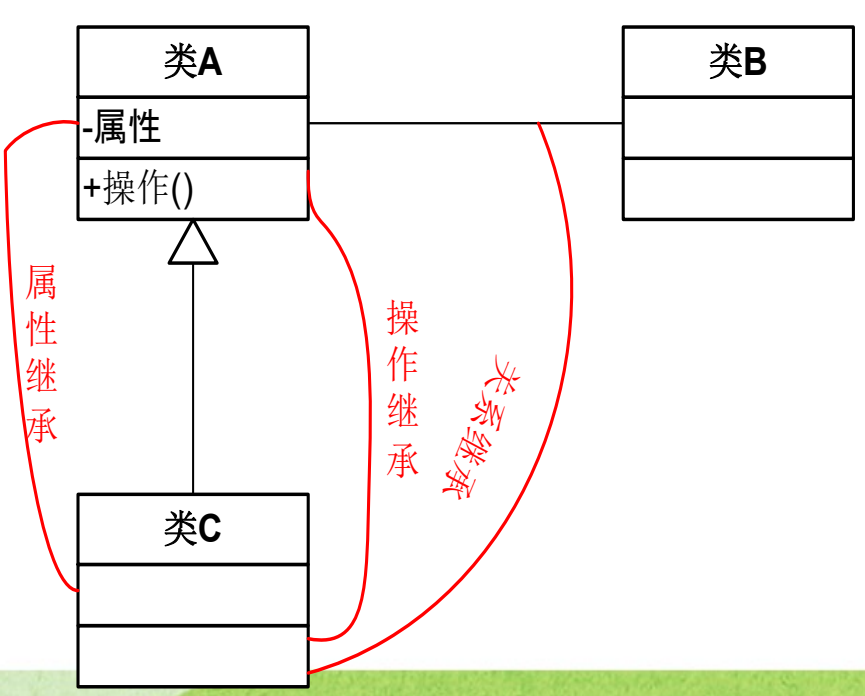
- 持有N元关联的类之间的关联信息的类称为N元关联类。
  - N元关联类和表征N元关联的菱形框之间用虚线进行连接。
  - 同普通关联类一样，在实现阶段要把概念意义上的关联类转化为普通类。

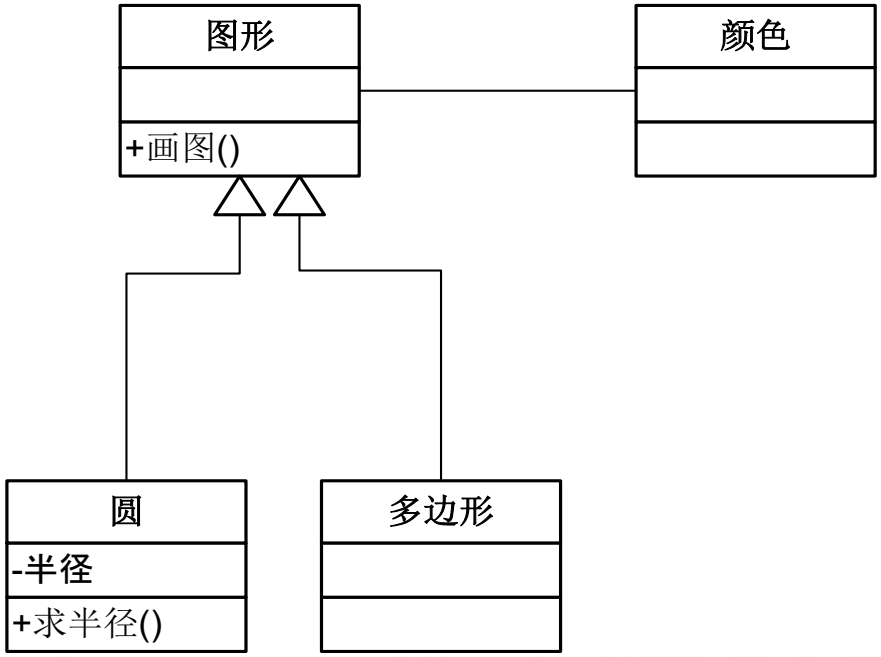




泛化

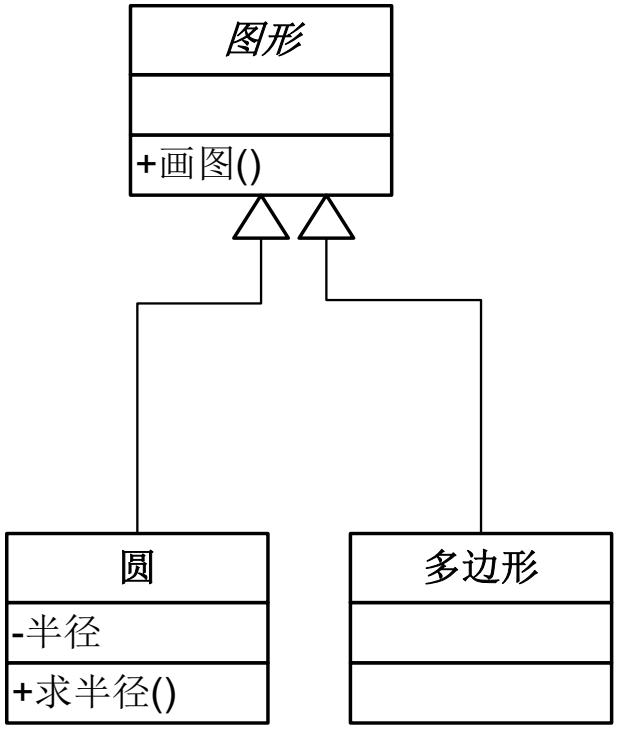
- 泛化表征的是类之间的继承关系。
  - 用空心三角连接，被继承的类称为超类（或父类），继承之后形成的新类称为子类。
  - 子类和超类之间是“is a”的关系。
  - 子类继承超类的所有性质，包括属性，操作和超类与别的类的之间的关系。
  - 子类可定义超类没有的新属性，新操作和新关系，还可重置超类中定义的某些操作。

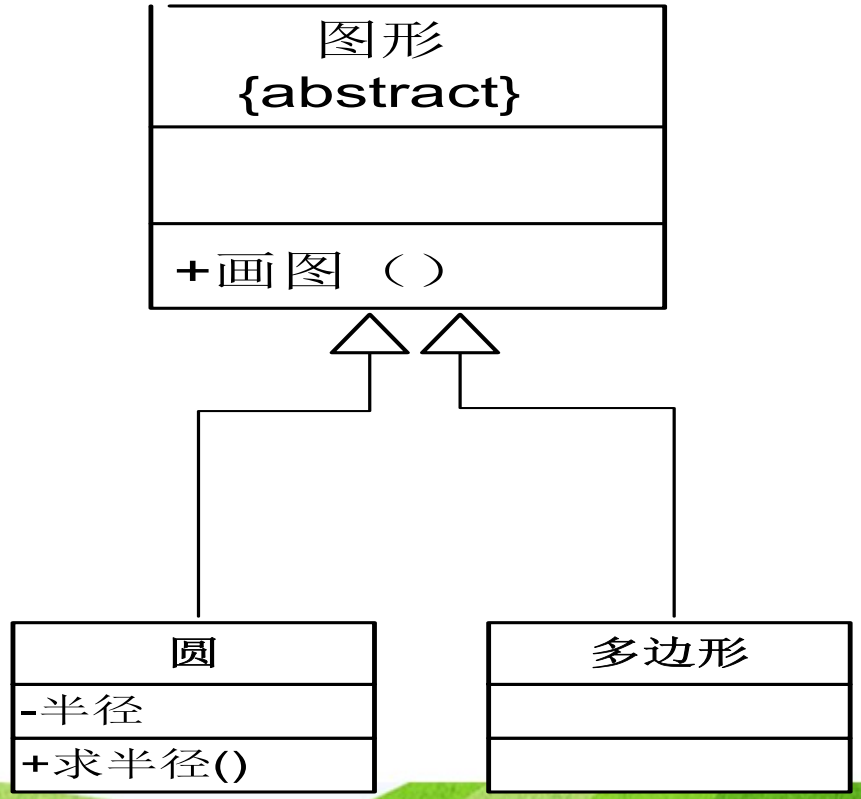




抽象类

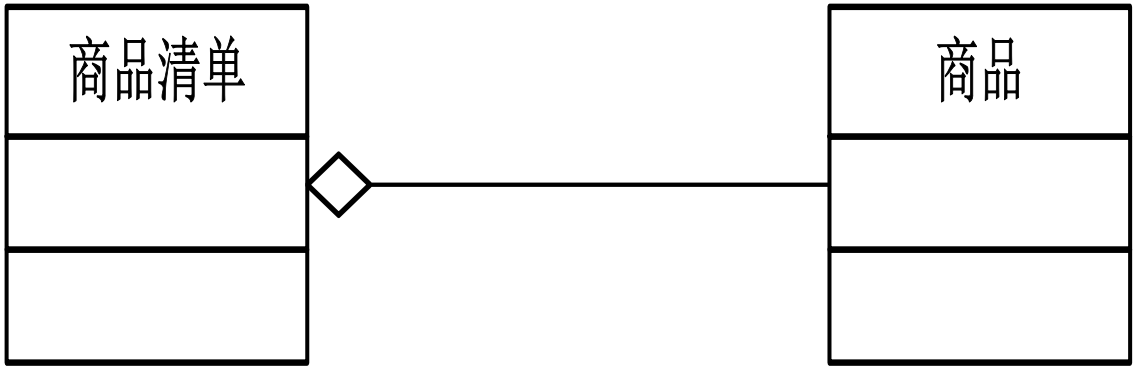
- 只表示概念的定义，总结子类的共通特性，不能生成实例对象的类。
  - 必须具有子类。
  - 类名用斜体字来表示。也可以用在类名下用特性{abstact}来标识。





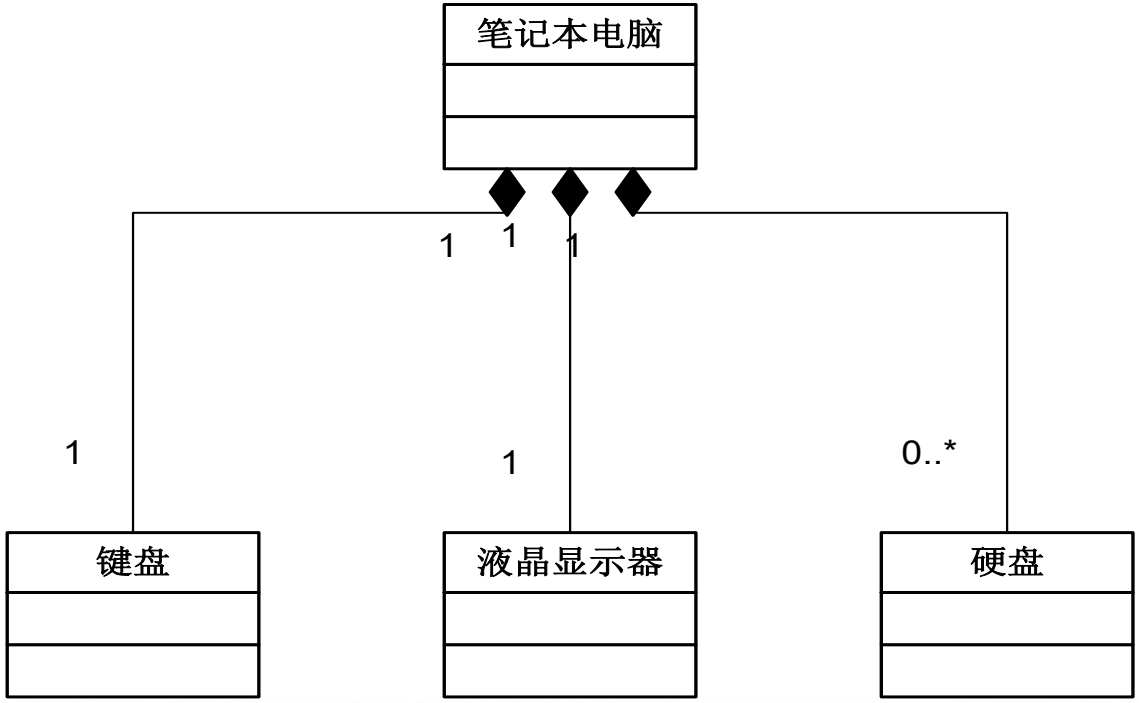
聚合（aggregation）

- 表示两个类之间的整体与部分（part-of）的关系。
  - 用空心菱形框来连接。
  - 两个类之间首先要具有关联关系，其次还要具有整体与部分的关系。
  - 在聚合关系的两端也可加上多重度关系。

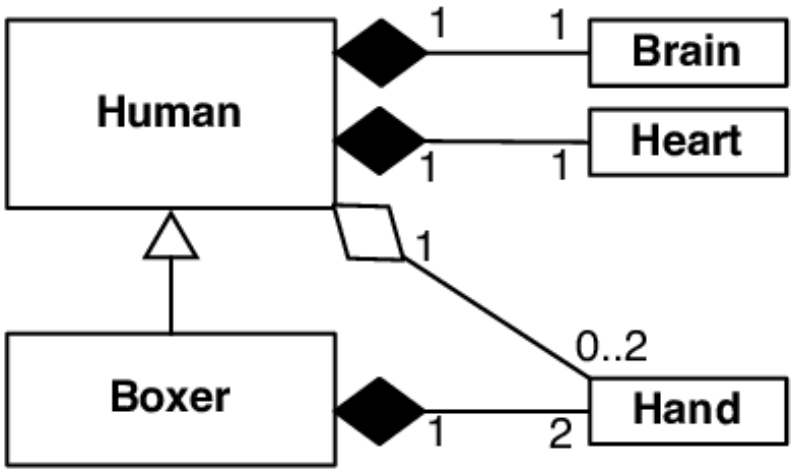


组合(composition)

- 组合是聚合的一种特殊形态，表征整体侧类的对象强烈拥有部分侧类的对象。
  - 整体侧类的实例的生命周期和部分侧类的实例的生命周期基本一致。
    整体侧对象生成，部分侧对象也随即生成；整体侧对象消失，部分侧对象也随即消失。
  - 组合关系用实心菱形框来表示。
  - 组合关系的两端也可指明多重度关系，与聚合关系不同的是，组合关系的整体侧的多重度一定为1。



- 组合跟聚合几乎相同，唯一的区别就是“部分”不能脱离“整体”单独存在，就是说，“部分”的生命期不能比“整体”还要长。
- 关联、聚合、组合关系是一般都是使用实例变量实现的. 从java 语法上无法区分，只有语义上的区别。



强弱程度依次为：组合>聚合>关联>依赖。

识别组合的途径

1）物理上整体的事物与它的部分事物，例如：人体与各种人体器官

2）组织机构与它的下级组织或部门，例如：市场部与公关部、产品部

3）团体（组织）与成员，例如：班级与学生

4）空间上的包容关系，例如：教室与桌椅

5）抽象事物的整体与部分，例如：学科与学科分支

6）具体事物和它的某个抽象方面，例如：人员基本信息与各个阶段的信息

7）在材料上的组成关系，例如：面包与面包材料

筛选组合的原则

1）**组合**中的整体对象和部分对象，若不属于问题域，则应该去掉，例如：业务系统中，职工与家庭的聚合关系。

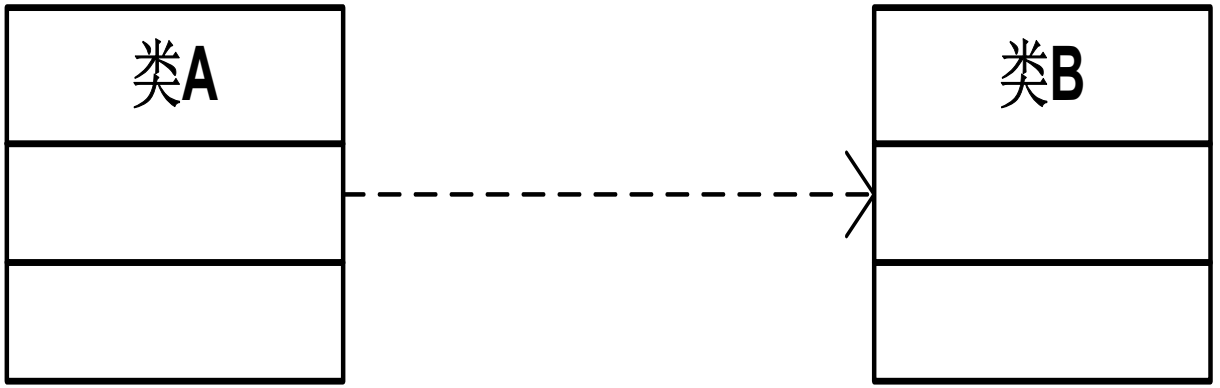
2）**组合**中的整体对象和部分对象，若不是系统责任需要的，则应该去掉，例如：赛车系统中，车和车轮的聚合关系。

3）**组合**中的部分对象若只有一个属性，则应该考虑取消它，把属性放到整体对象中去。例如：汽车销售系统中，汽车和只有规格一个属性的车轮聚合。

4）如果两个对象不能明显分出整体---部分关系，则不应该用**组合**关系表示。例如：财务系统中，名细帐和总帐，不明显有整体部分结构。

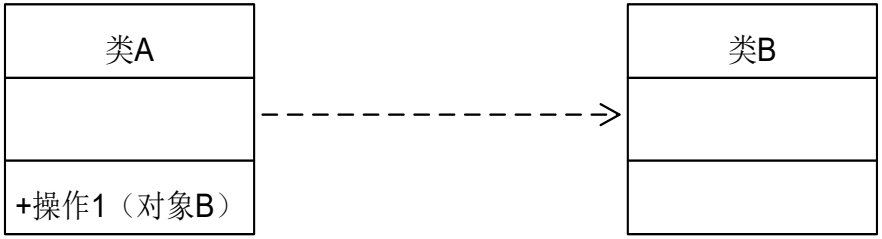
依赖关系

- 依赖关系表示的是两个类之间的一种使用上的关系。
  - 用虚线箭头链接具有依赖关系的两个类。箭头指向被使用的类。

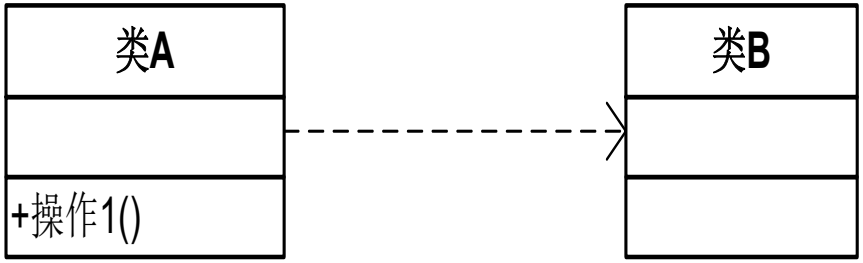


依赖关系

- 类之间的依赖关系有以下三种：
  - 一个类的对象作为另外一个类的参数被调用。

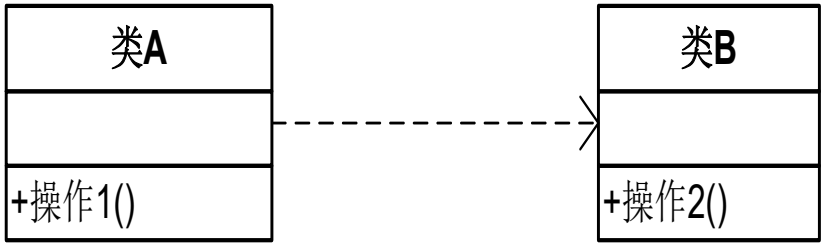


  - 一个类的对象作为局部变量被调用。




```
操作1（）{
类B 对象B;    //做成类B的对象
  ，
  ，
}
```

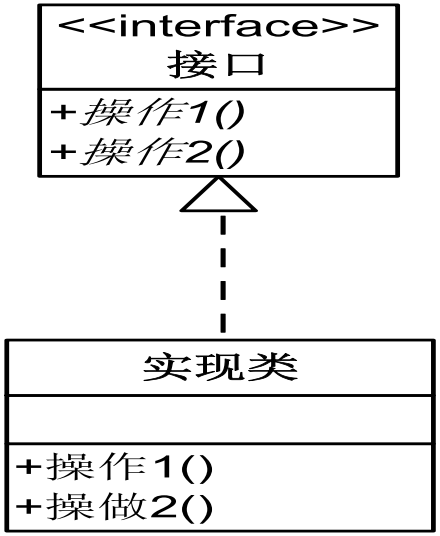
  - 一个类的对象作为全局变量被调用。

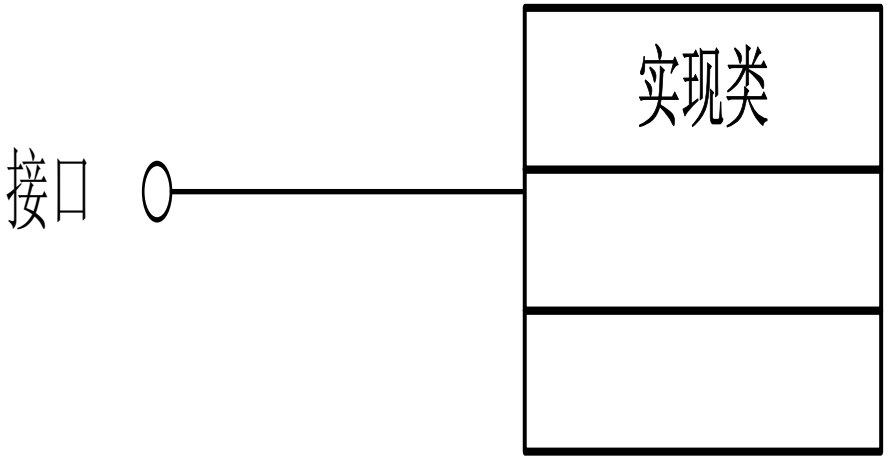



```
操作1（）{
对象B.操作2;     //调用类B的操作
  ，，
}
```

接口

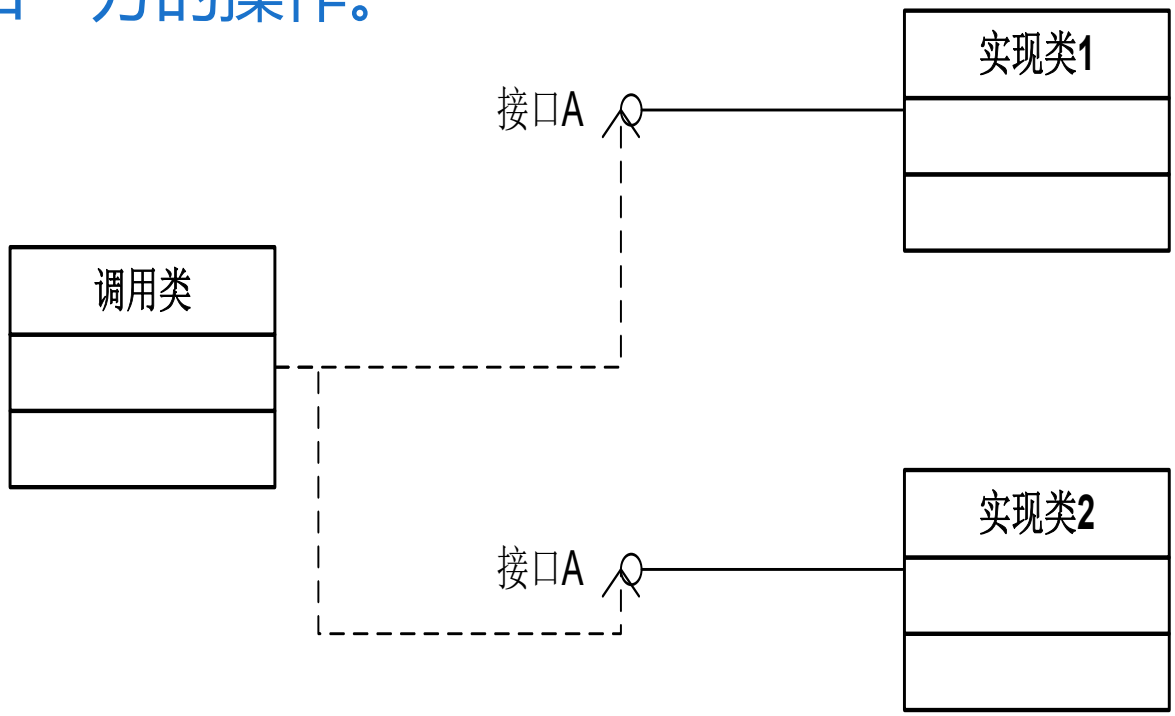
- 定义类或者组件，或者子系统公开给外部的操作的功能。
  - 只定义操作的功能，不对操作进行实现。接口不可实例化。
  - 实现接口中定义的操作的类叫实现类，和接口之间通过空心三角连带虚线(实现关系)进行连接。也可用圆圈（表示接口）加实线来连接接口和实现类。
  - 接口中没有属性，只有操作，而且可以根据需要在接口中定义多个操作。
  - 实现类可以实现多个接口。





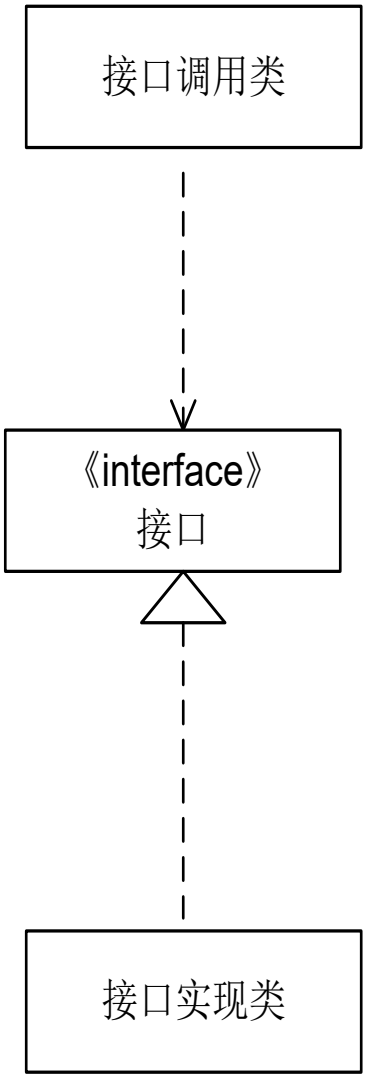
接口

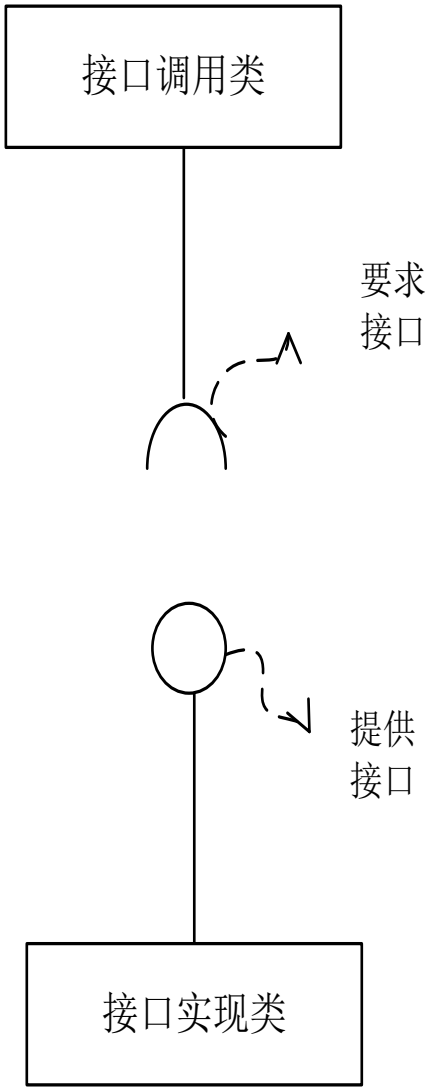
- 定义接口的优点：
  - 只要接口定义不变，即使对接口的实现进行改变，也不用改变调用接口一方的操作。

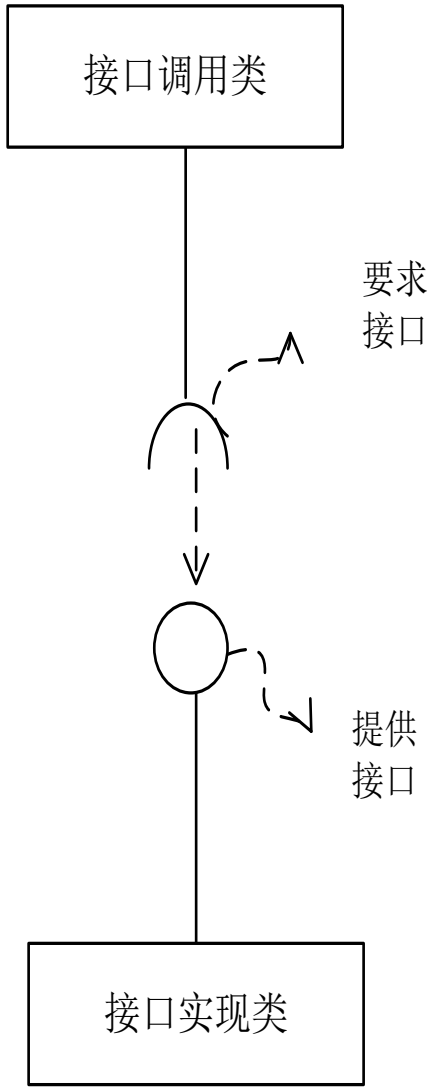


实现类1和实现类2中，对于接口A的实现不管如何改变，只要接口A的定义不变，对于调用类来说没有任何影响，不用改变其接口调用方法。

接口访问的表示方法


接口调用类
《interface》
接口
接口实现类


接口调用类
要求
接口
提供
接口
接口实现类

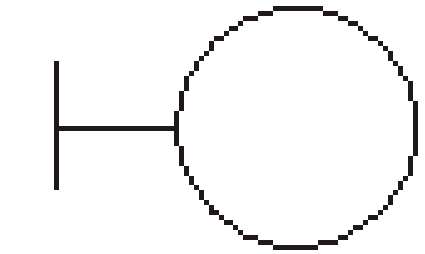

接口调用类
要求
接口
提供
接口
接口实现类

边界类、控制类和实体类的划分

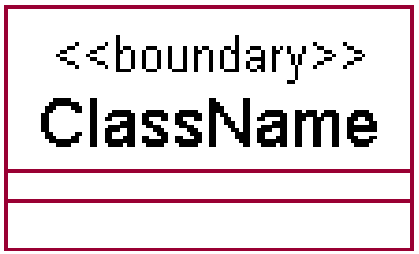
◆UML中有三种主要的类版型：**Boundary，Entity，Control**。

◆**边界类(Boundary class)**位于系统与外界的交界处，包括所有窗体(form)、报表(report)、与外部设备的接口(interface)，以及与其它系统的接口等。

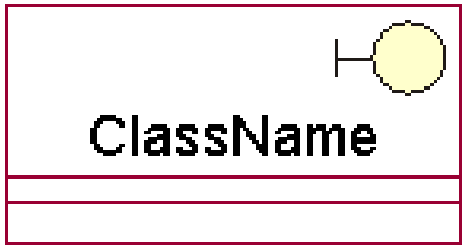
◆UML中边界类的表示：



Icon形式



Label形式

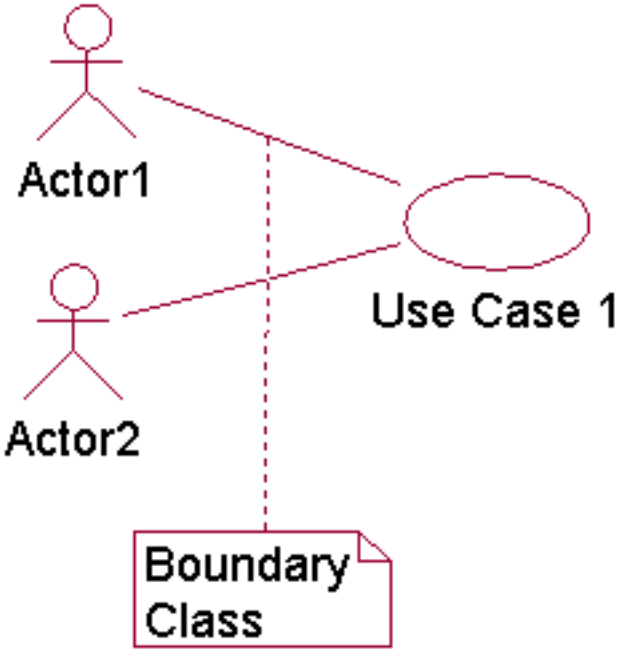


Decoration形式

说明：

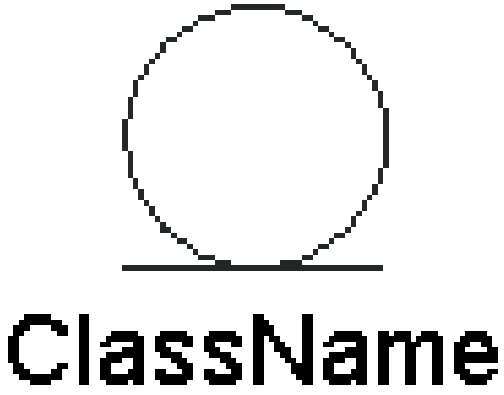
◆通过use case图可以确定需要的边界类。

◆每个actor/use case交互至少要有一个边界类。但并非每个actor/use case对要生成唯一边界类，多个actor启动同一use case时，可以用同一个边界类与系统通信。例如：

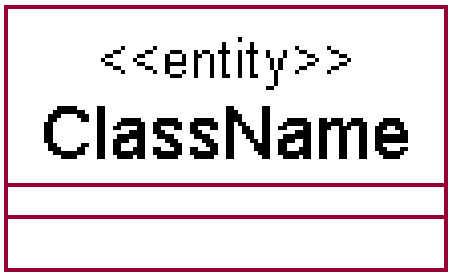


◆**实体类(Entity class)**保存要放进持久存储体的信息。

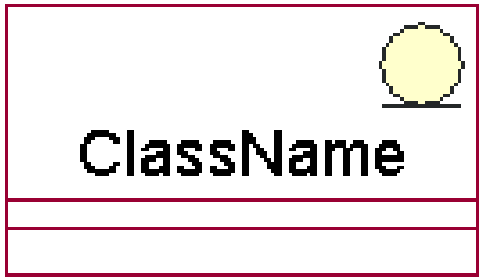
◆UML中实体类的表示：



Icon形式



Label形式



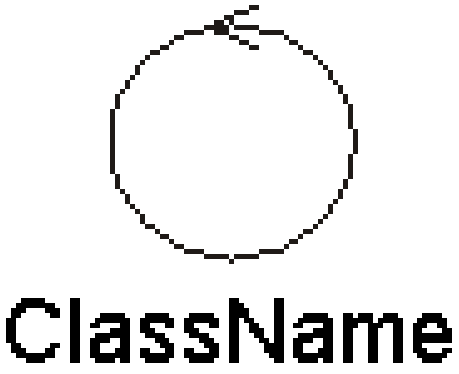
Decoration形式

说明：

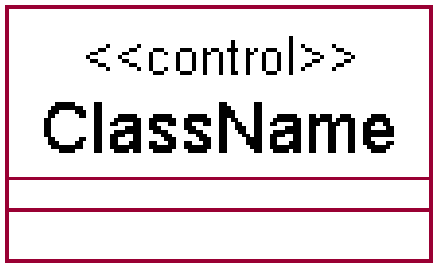
- 通常，每个实体类在数据库中有相应的表，实体类中的属性对应数据库中表的字段。
- 实体类可以通过事件流和交互图发现。实体类通常用领域术语命名。

◆**控制类(Control class)**是负责其它类工作的类。

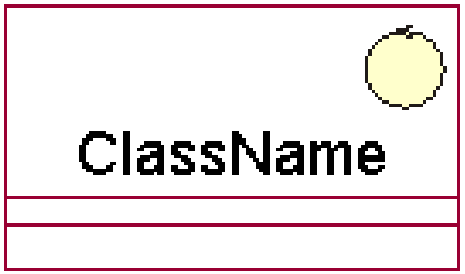
◆UML中控制类的表示：



Icon形式



Label形式



Decoration形式

说明：

- 每个use case通常有一个控制类，控制use case中的事件顺序。
- 控制类也可在多个use case间共用。
- 其它类并不向控制类发送很多消息，而是由控制类发出很多消息。

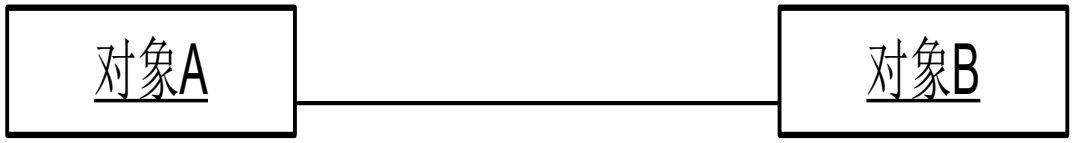
8.3 对象图（object diagram）

- 类图表示类和类与类之间的关系，对象图则表示在某一时刻这些类的具体实例和这些实例之间的具体连接关系。

对象图可以看作类图的实例，帮助人们理解一个比较复杂的类图。对象图也可用于显示类图中的对象在某一时刻的连接关系。

8.3对象图-对象建模的表示

- 对象图表示对象之间的关系。在考虑类图的多重度或者系统在某一时刻的状态时使用。
- 对象图的组成元素：
  - 对象
  - 链



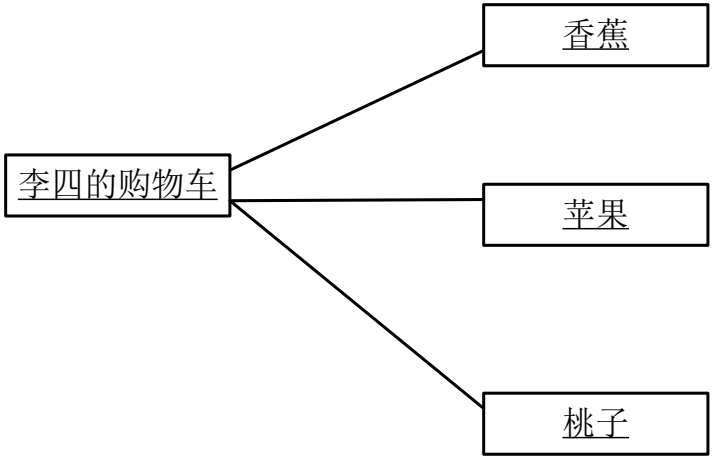
- 对象图中对象的表示方法：
  - 矩形框中写对象名，对象名下有下划线。
  - 对象有以下三种表示方法：



●只表示对象名　　●类名和对象名都表示　　●只表示类名

- 对象图中链的表示方法：
  - 两对象之间用实线进行连接。

- 对象图具有以下用途：
  - 是系统的快照，表示系统在某一时刻的状态。



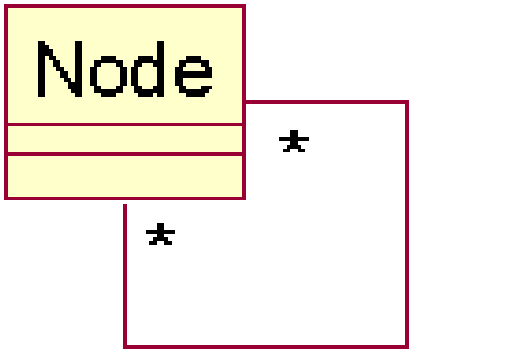
●在某一时刻，系统中“顾客李四的购物车”里有“香蕉”“苹果”“桃子”三件商品。到了另一时刻，该购物篮里的商品对象可能会有所变化。

  - 决定类图中的多重度关系。

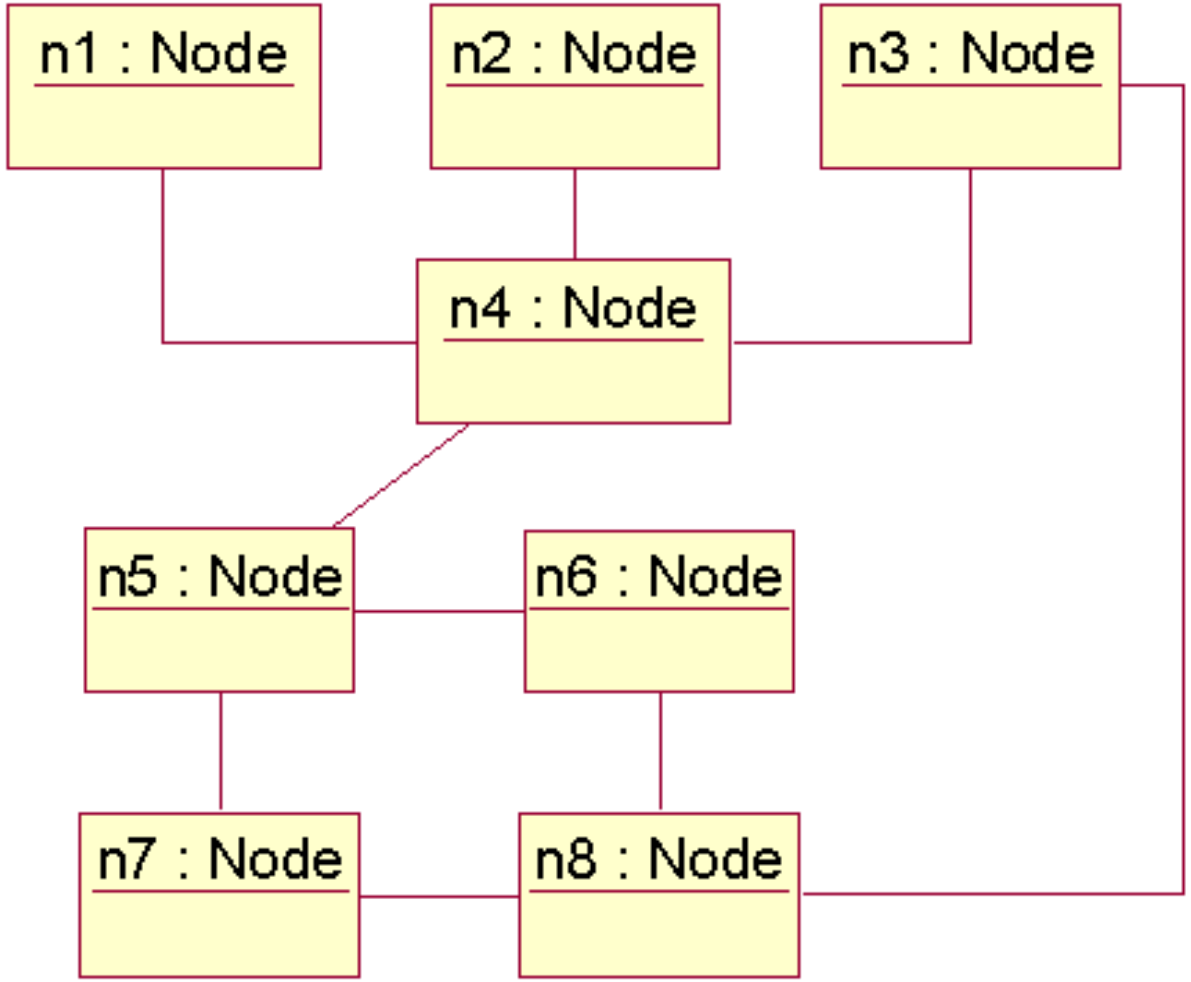


8.3对象图-例子

◆例：表示网络间节点关系的类图及其一个对象图的例子。

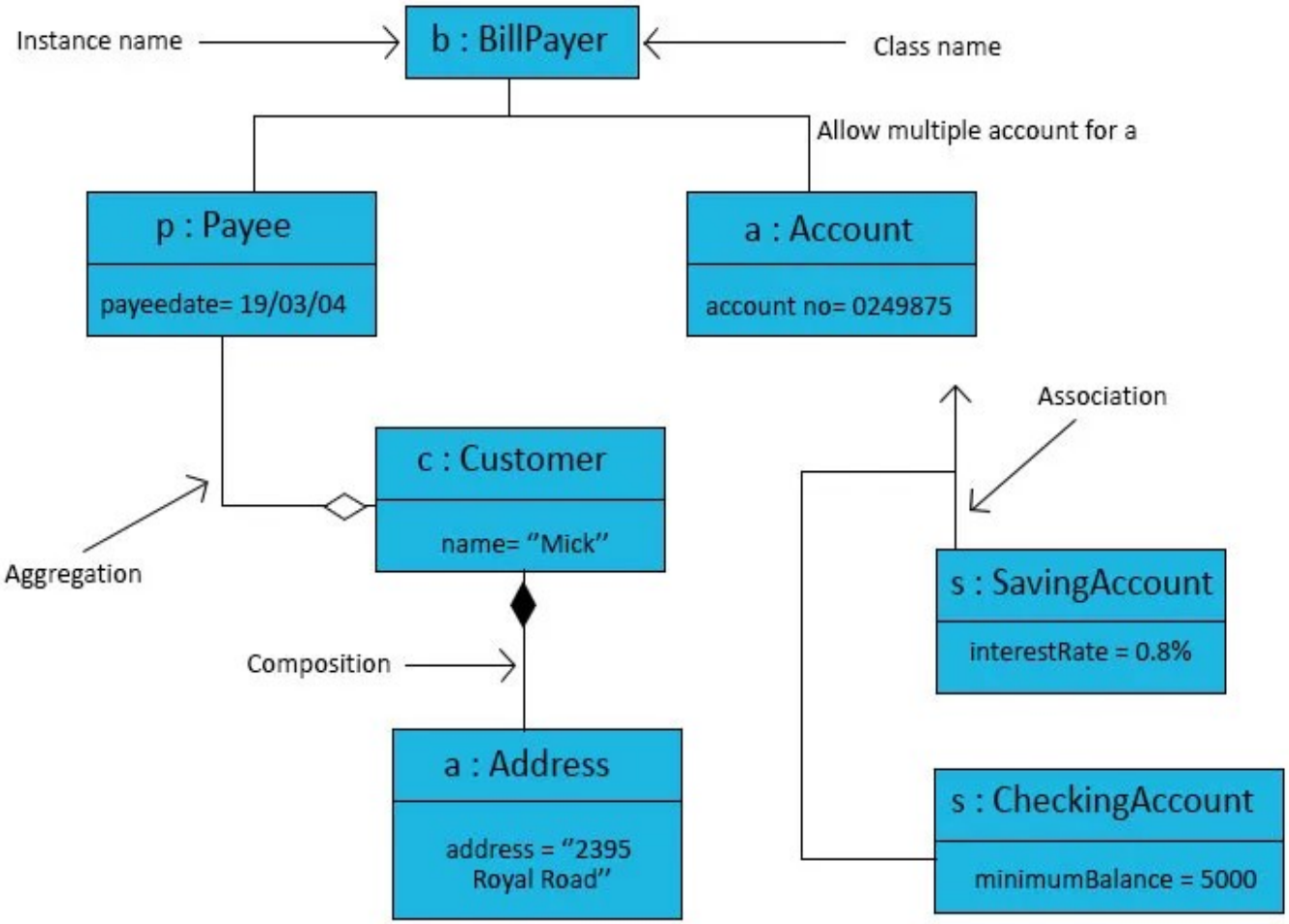


类图

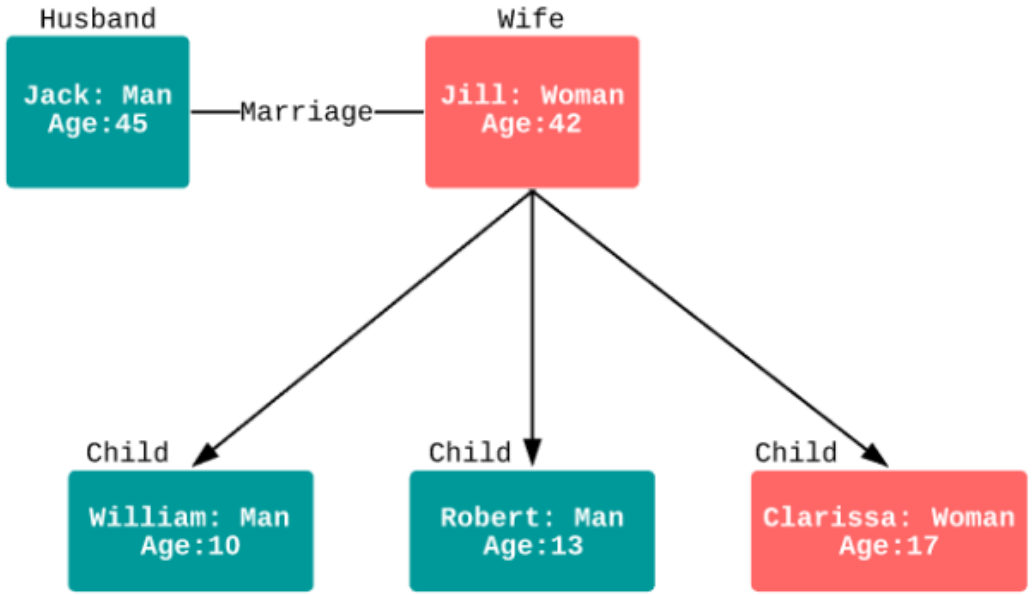


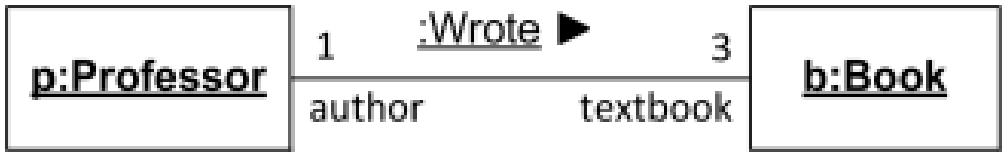
对象图

◆UML 2.4规范中，对象图也可以用类图同样的关联符号；也有人资料画的对象名字没有下划线。



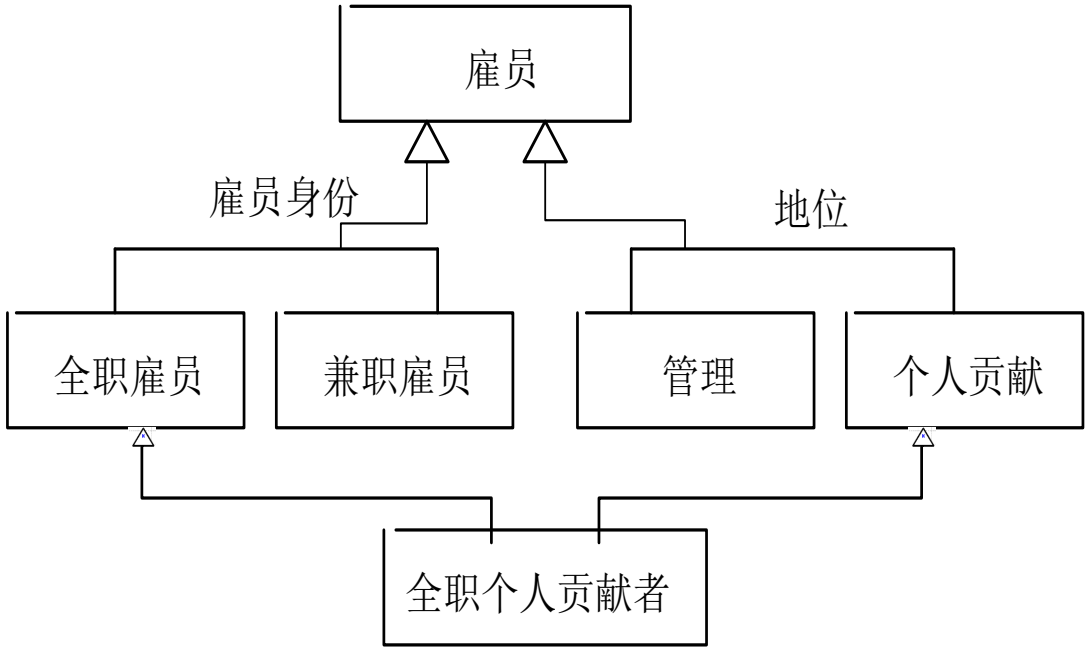
◆UML 2.4规范中，对象图也可以用类图同样的关联符号；也有人资料画的对象名字没有下划线。

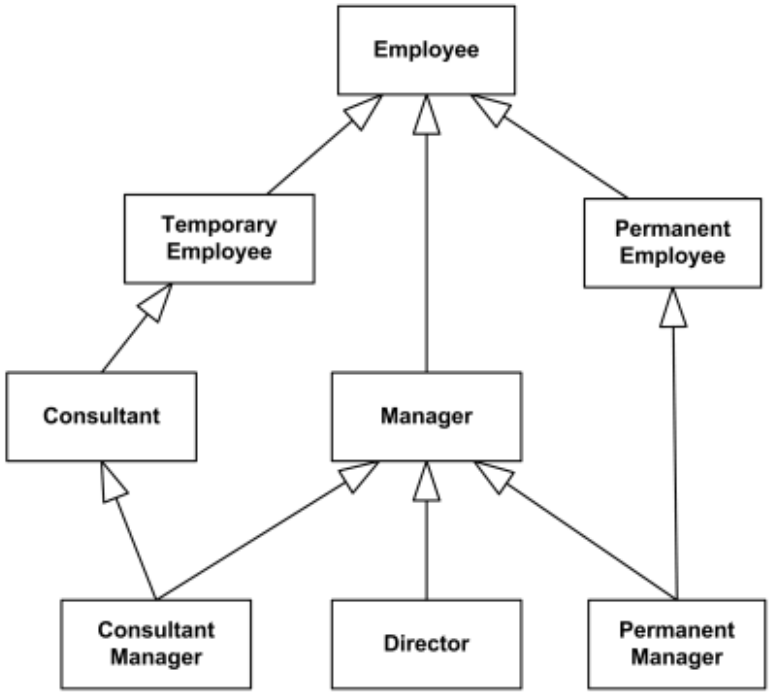




8.4多重继承

- 多重继承最常见的是无交集的类集合，即子类继承每个无交集父类的一个。例如，图8.26中，雇员按照身份和地位分别分为全职雇员、兼职雇员，管理和个人贡献，它们都是雇员的子类。全职个人贡献者既是全职雇员，也是个人贡献，合并了2者的特征。在这个类图中，还可以扩展出更多的多继承组合，例如全职管理、兼职个人贡献、兼职管理。这些组合的形式依赖于应用的需求。

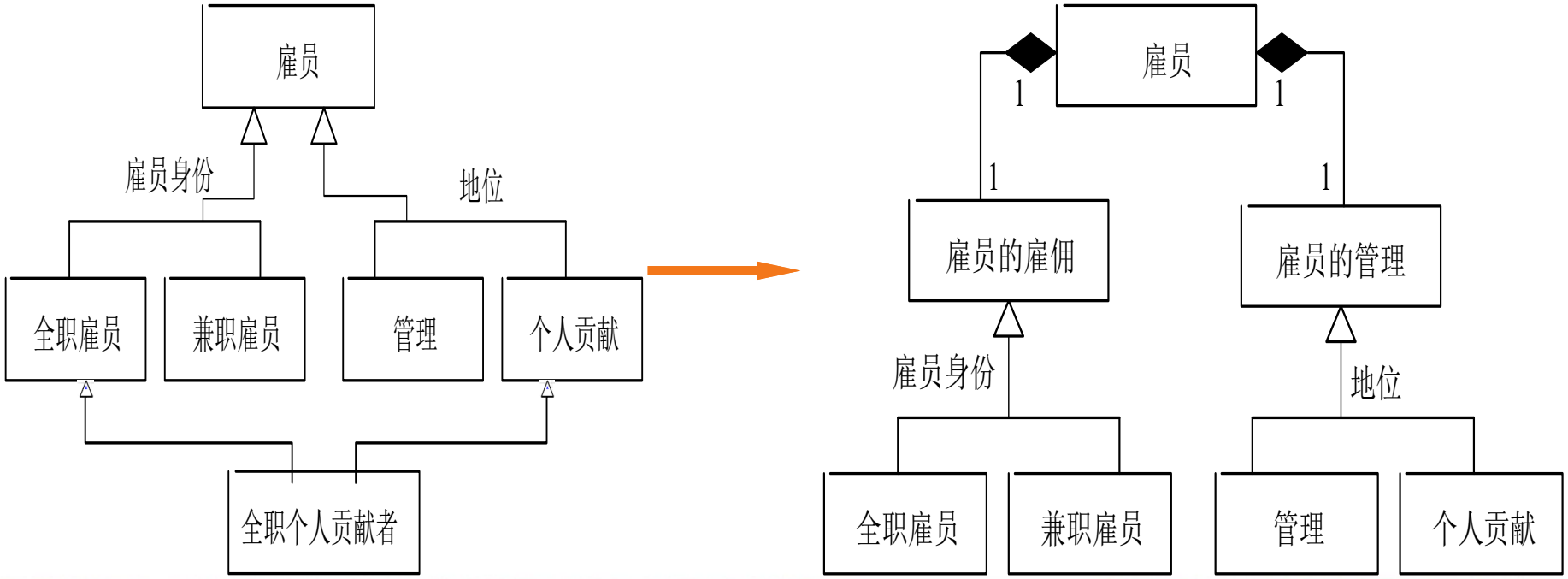




8.4多重继承-使用部件组合的委托

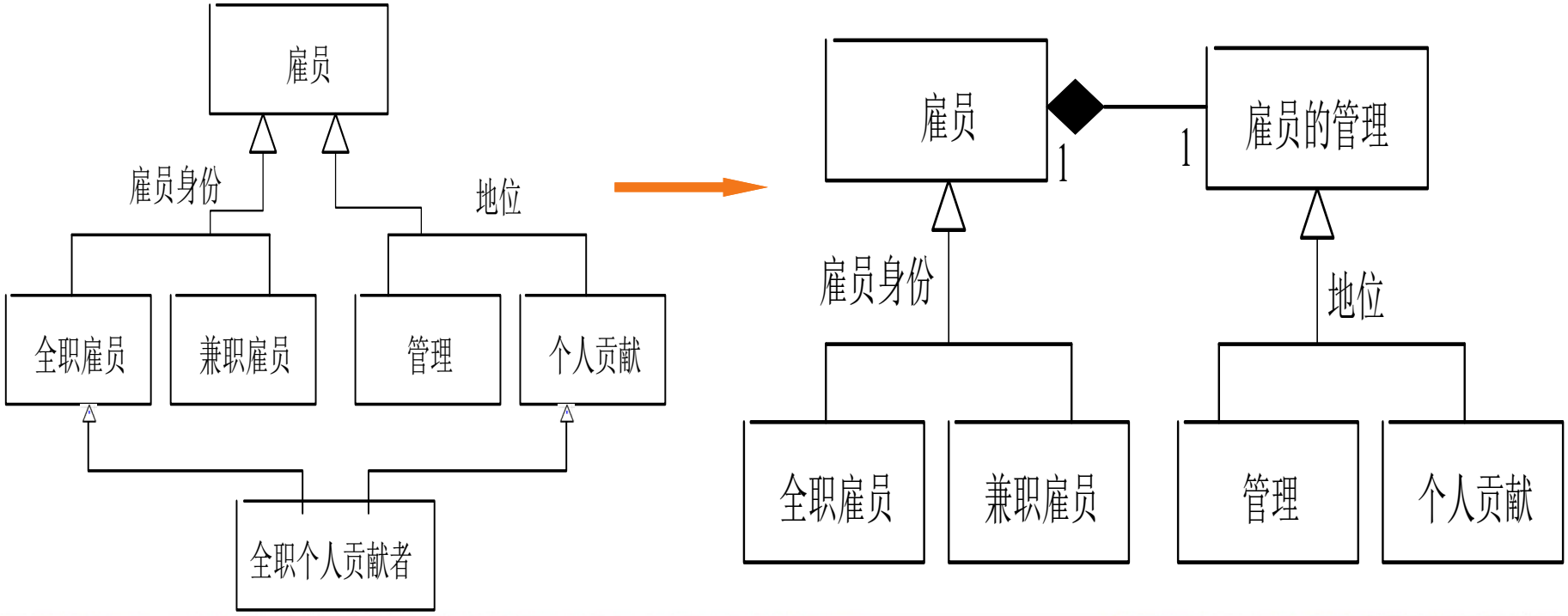
- 可以将多重继承中的父类重新构造成组合的方式，其中组合的每个部件替换一个泛化。这种方法把具有唯一ID的单个对象替换为一组相关对象，这组对象成为一个扩展的对象。此时，组合之间的操作就不能自动捕捉，需要组合来捕捉操作，并委托给适当的部件。

对上一个例子，使用部件组合的委托方式，就得下图的实现



8.4多重继承-继承和委托相结合

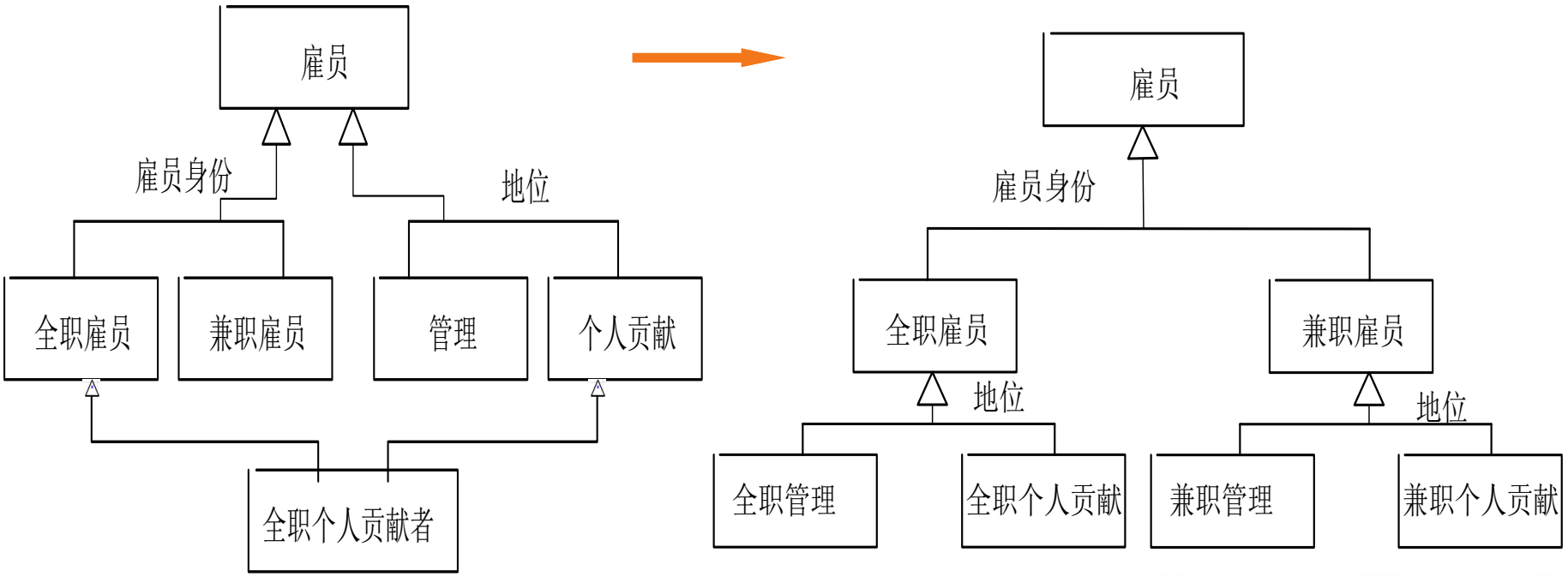
- 对于上一页的多重继承，把主要的类继承下来，把次要的一个继承使用委托来实现。如下图所示。把多重继承中管理类和个人贡献类作为一个组合来处理，即子类继承自全职雇员或者兼职管理，并且组合一个管理类或者个人贡献类，这样的话，就达到多重继承的目的。



8.4多重继承-嵌套泛化

- 嵌套泛化采用先分解一个泛化，再这泛化的基础上分解另外一个泛化。这种方法的思想是列出所有可能的组合。

对上一页的例子，先按照雇员身份将子类分解为全职雇员和兼职雇员。再分别多全职雇员和兼职雇员类按照地位进行泛化，得到全职管理、全职个人贡献、兼职管理以及兼职个人贡献类，这样所有的可能多重继承都可以得到满足。



绘制对象图的过程

- 论证类模型的设计：当设计了类模型时，你可以通过对象图来模拟出一个运行时的状态，这样就可以研究在运行时设计的合理性。同时，也可以作为开发人员讨论的一个基础。
- 分析和说明源代码：由于类图只是展示了程序的静态类结构，因此通过类图看懂代码的意图是很困难的。因此在分析源代码时，可以通过对象图来细化分析。而对于开发人员，对于逻辑较复杂的类交互时，可以考虑画出一些对象图来做补充说明

- 链接(link)：是对象之间物理上或者概念上的连接。
  如：张三为一个公司工作(workfor)，workfor就是一个连接
- 关联（Association）描述了有着共同的结构和语义的一组链接。表示两个类之间存在某种语义上的联系。关联关系提供了通信的路径，它是所有关系中最通用、语义最弱的。

类图的抽象层次

- 在软件开发的不同阶段使用的类图具有不同的抽象层次。一般地，类图可分为三个层次，即**概念层**，**说明层**和**实现层**。
- 类的**概念层**，**说明层**和**实现层**的划分最先是由Steve Cook和John Daniels引入的。

◆**概念层(Conceptual)**类图描述应用领域中的概念，一般地，这些概念和类有很自然的联系，但两者并没有直接的映射关系。

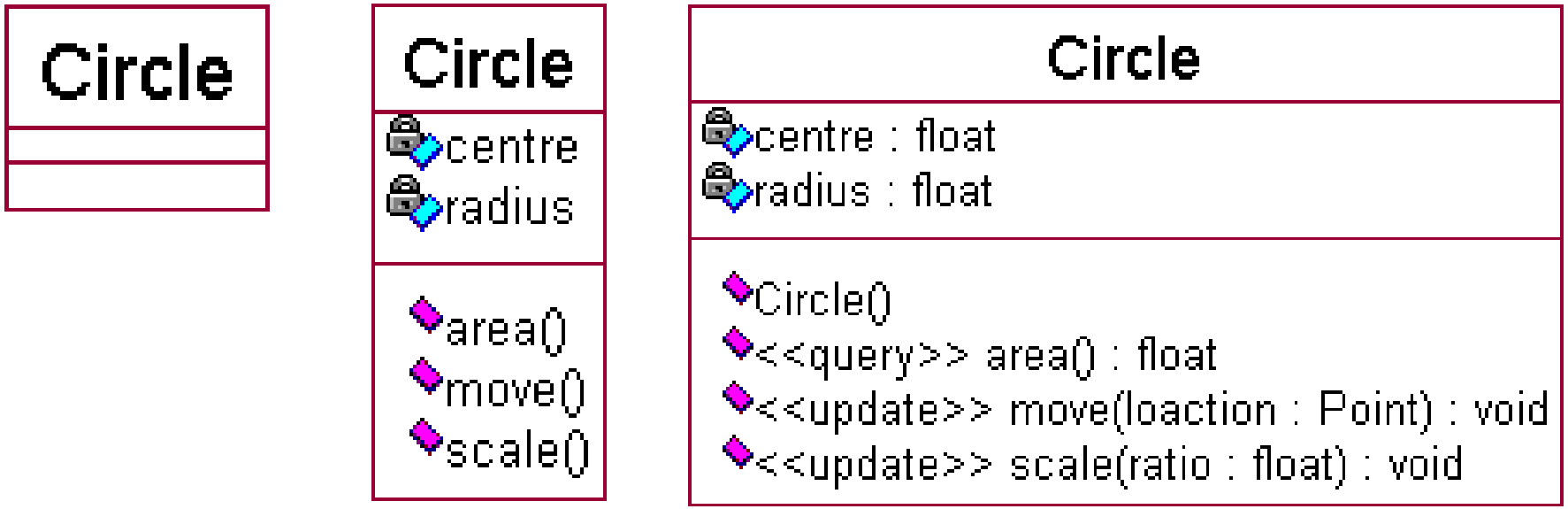
◆画概念层的类图时，很少考虑或不考虑实现问题，因此，概念层的类图应独立于具体的程序设计语言。

◆**说明层(Specification)**类图描述软件的接口部分，而不是软件的实现部分。

◆**实现层(Implementation)**类图才真正考虑类的实现问题，揭示实现细节。

■实现层的类图可能是大多数人最常用的类图，但在很多时候，说明层的类图更易于开发者之间的相互理解和交流。

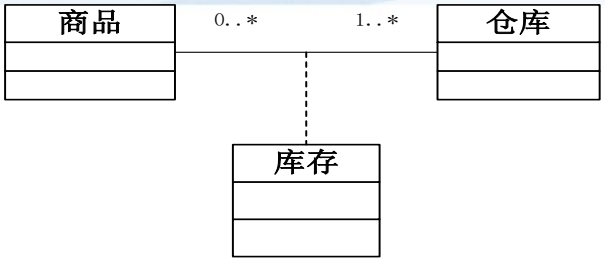
例：类图的三个层次的例子。

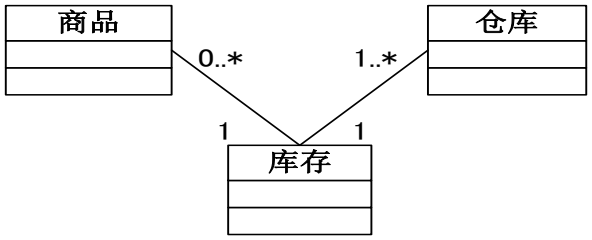


概念层　　说明层　　实现层

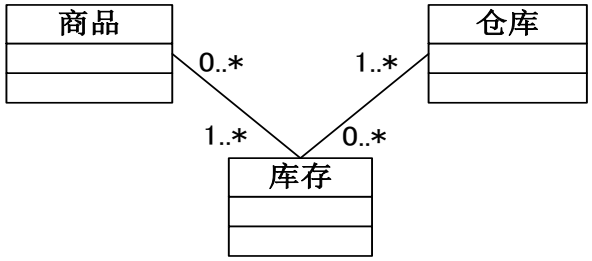
例题1

与以下类图等价的类图是

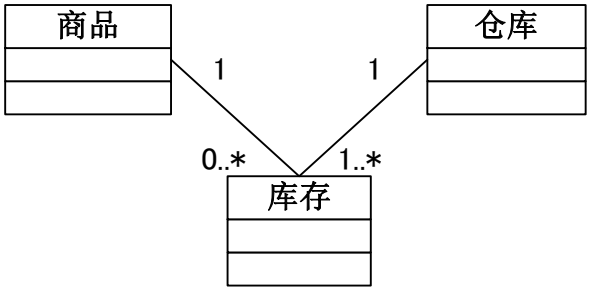




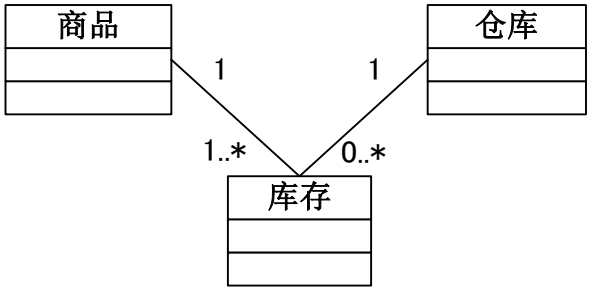
a



b



c



d

例题2

选出是类和对象关系的描述。

1、形状和三角形

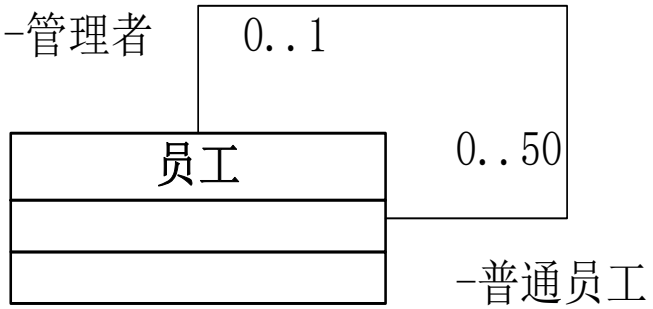
2、动物和鸟

3、营业员和营业员小王

4、课程表和课程

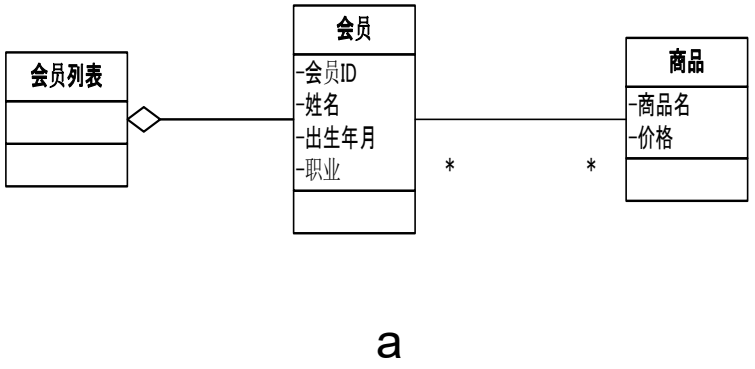
例题3

关于下面的类图，说法正确的是：

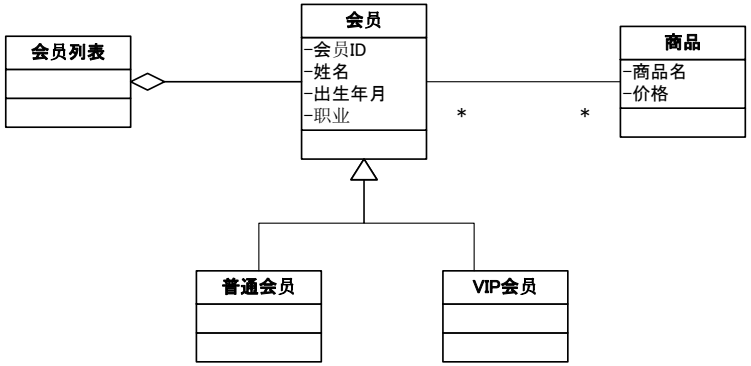


1、一个员工可能管理多个员工，也可能一个员工也不管理
2、管理者除了能管理普通员工外还可以管理管理者
3、每个员工都对应一个管理者
4、一个员工最多可能管理50名普通员工

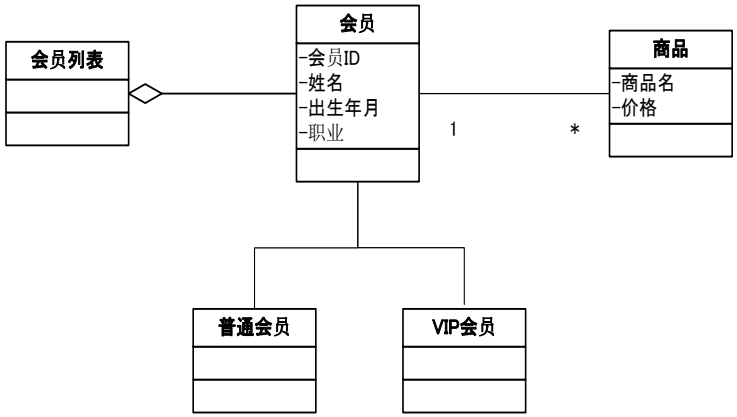
例4.根据描述选择正确的类图：某公司的网上购物系统，采用会员制进行管理。有一个会员列表来记录所有会员的信息；会员的信息包括会员ID、姓名、出生年月、职业等。并且会员还分为普通会员和VIP会员。该系统还会记录每个会员所购买过的商品信息，商品的信息包括商品名、价格等



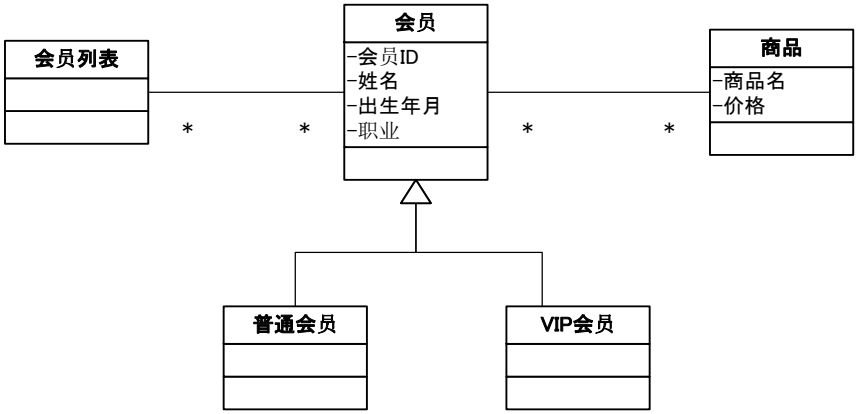
a



b



c



d

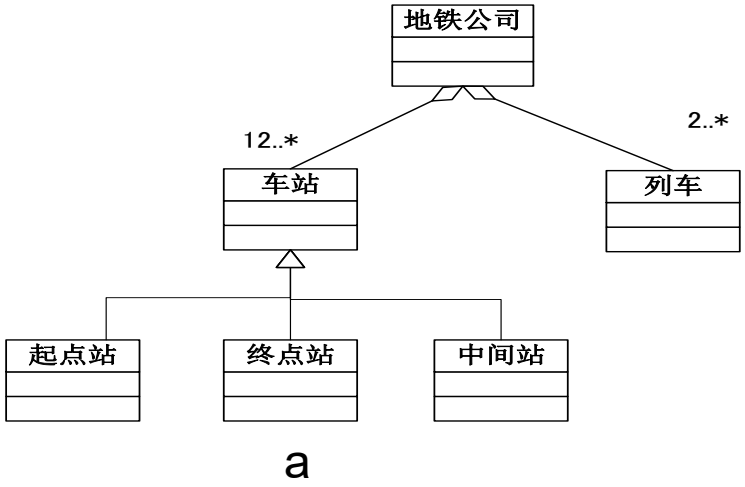
例题5

根据下面的描述画出类图

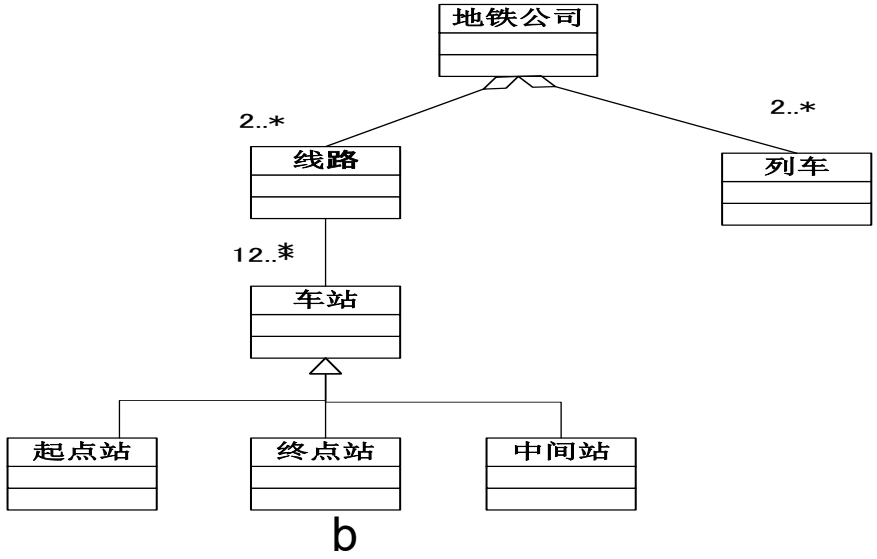
某公司的设备预约系统，预约者登录后从设备列表中选择要预约的设备。该公司大约有200到300台设备。可预约的设备包括笔记本电脑、投影仪等，但是为了保持资源合理利用同时最多只能预约4台设备。预约时需要填写使用的期间等信息。

例6.根据下面的描述选择最恰当的类图

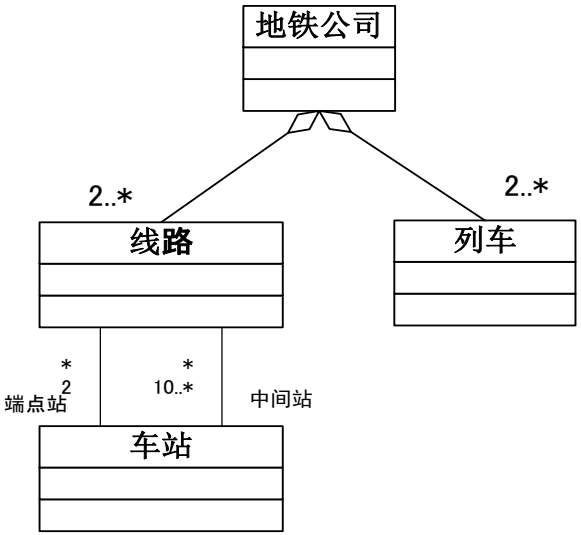
一个地铁公司有至少有2辆以上的列车和2条以上的线路，每条线路至少有10个中间站点，另外还包括一个起点站和一个终点站。



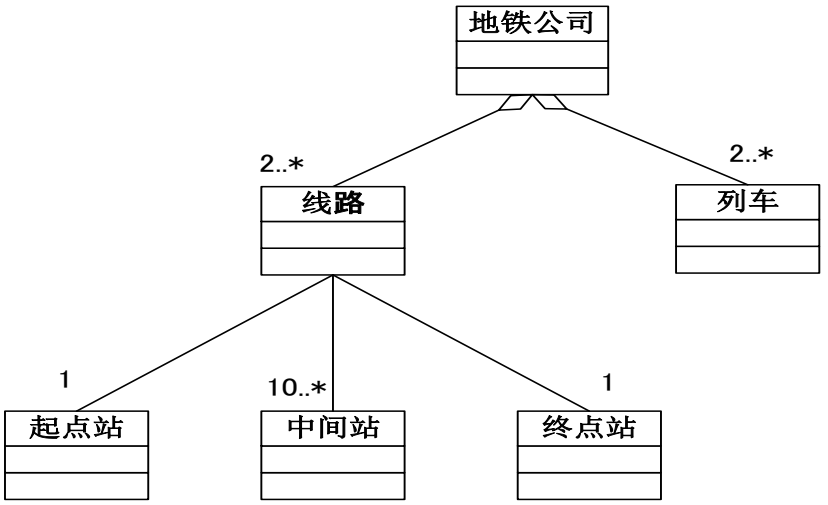
a



b



c



d

例7.用分析类图来表现某公司的联欢会和学习会的结构。请选择符合类图的（a）～（i）的选项，完成模型（一个选项可以选多次）。

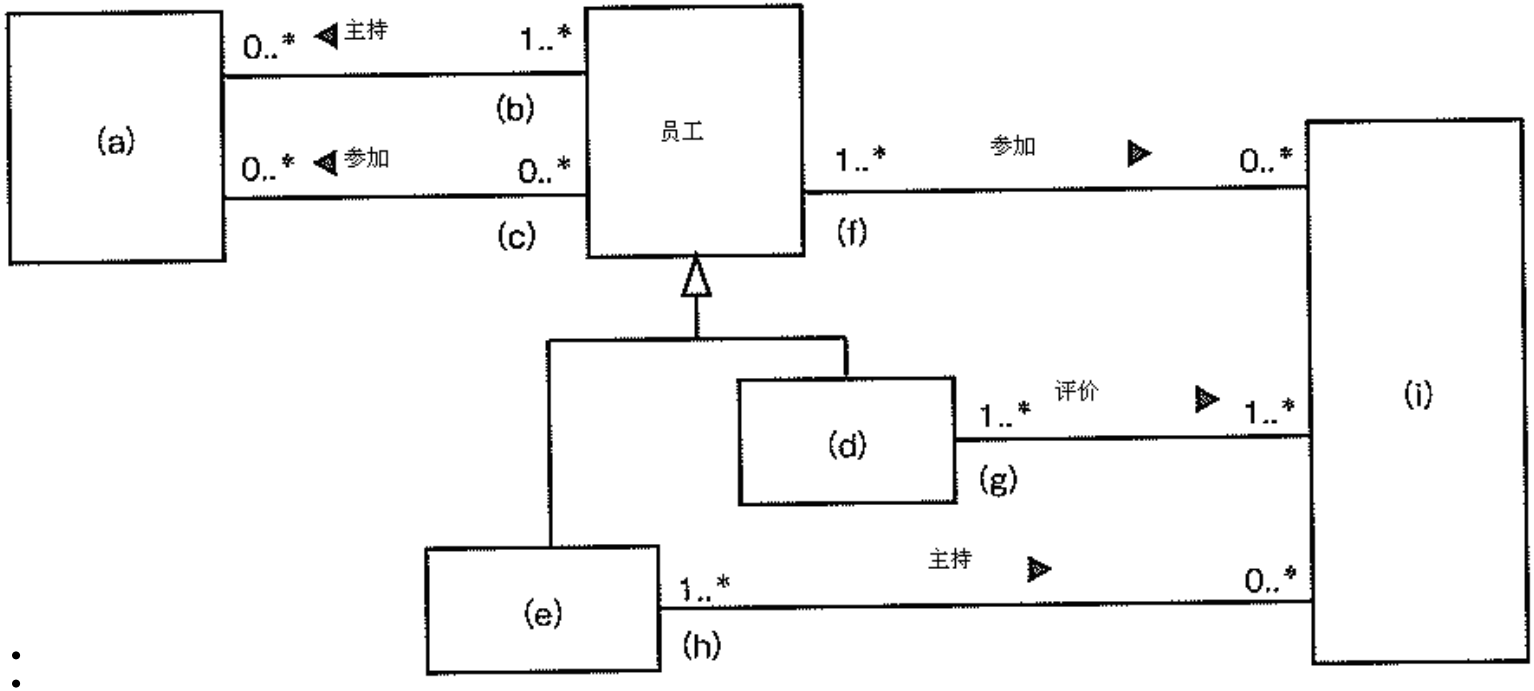
员工以参加者的身份参加联欢会和学习会。

员工也可能既不参加联欢会也不参加学习会。

联欢会和学习会必须至少有一名主办人，学习会通常由普通员工举办。

学习会由公司进行评价，管理层的员工为评价人。

规定管理层的员工必须对至少一个学习会进行评价。



选项：

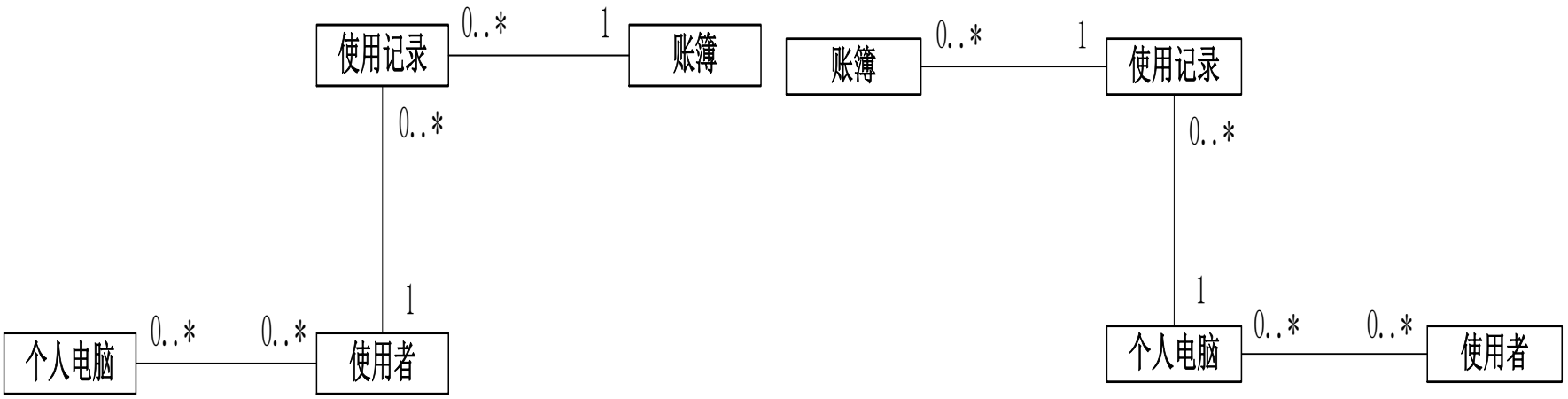
（1）联欢会　（2）学习会　（3）参加者　（4）主办人

（5）普通员工　（6）管理层员工　（7）评价人

例8.在某个市民中心里，对市民开放了个人电脑的使用。想使用个人电脑的市民，必须注册成为使用者。注册后就会发行使用者ID。

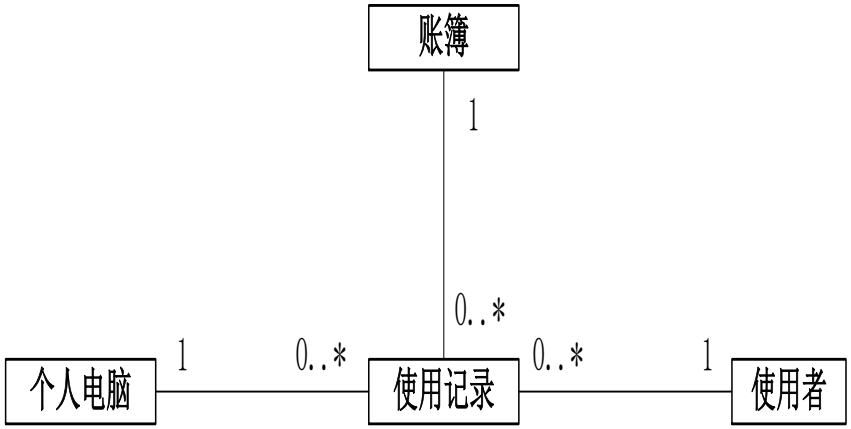
有多台个人电脑，使用者可以使用其中的任意一台。使用者也可以多次使用个人电脑。

使用者使用个人电脑时，必须填写使用记录。在使用记录中记下使用者ID。使用者用完个人电脑后把使用记录交给负责人，负责人把使用记录的信息输入到系统内的账簿中。负责人可以把个人电脑或使用者作为关键字，从账簿中查询出使用记录

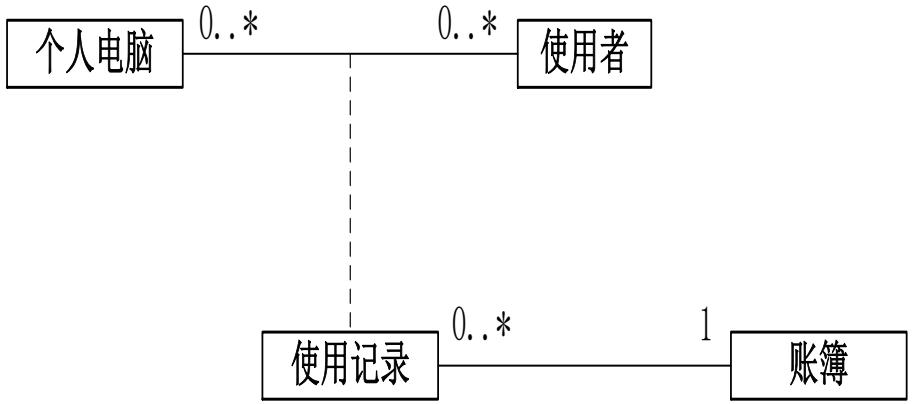

使用记录
0..*
1
账簿
0..*
个人电脑
0..*
0..*
使用者
1
账簿
0..*
1
使用记录
0..*
1
个人电脑
0..*
0..*
使用者

a

b


账簿
1
0..*
个人电脑
1
0..*
使用记录
0..*
1
使用者

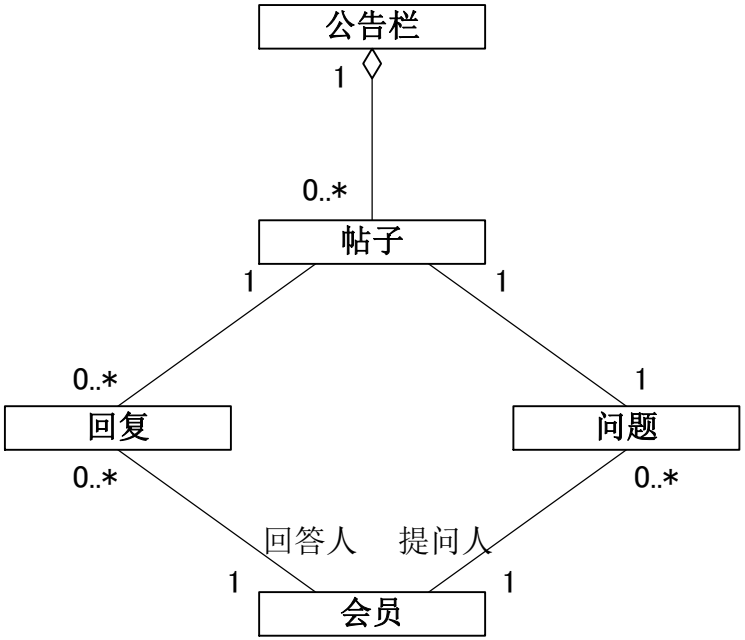
c


个人电脑
0..*
0..*
使用者
使用记录
0..*
1
账簿

d

例9.阅读以下描述后，请选择不符合类图的所有选项。这是互联网上的某个公告栏。因为是会员制的，所以提问人和回答人都必须是会员。

提问人新创建一个帖子后，写下问题。回答人在该帖子中写下回复。



A．在该模型中，不允许会员回答自己所提问的帖子。

B．在该模型中，也不允许提问的会员回答其它的帖子。

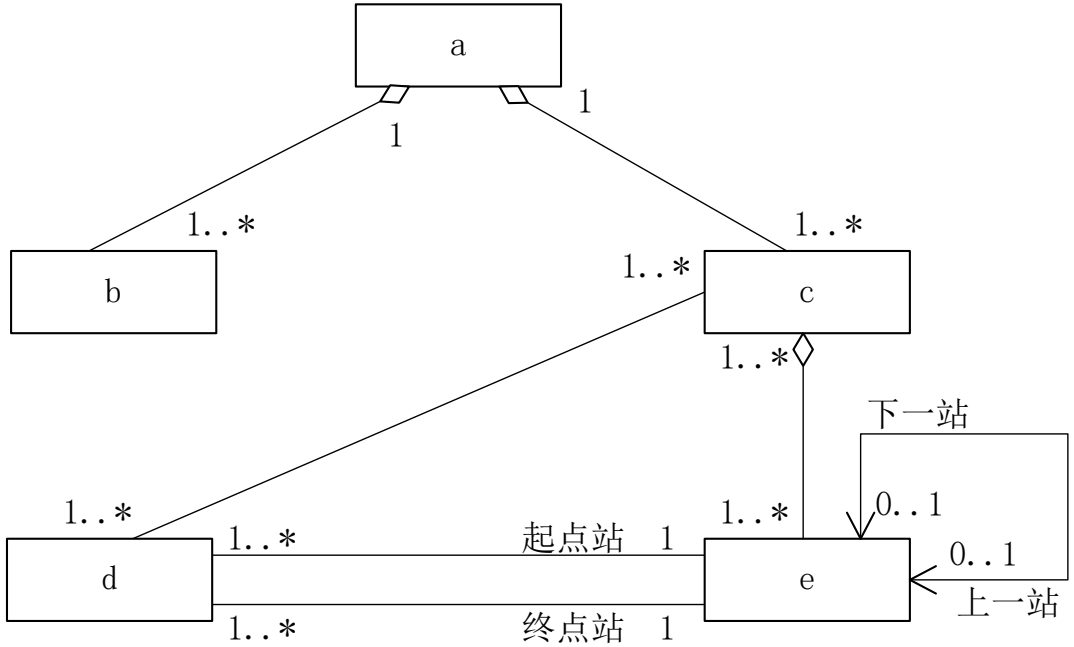
C．在该模型中，会员能对同一帖子回答两次以上。

D．在该模型中，一个会员对某帖子只能提问一次。但假如又新创建了帖子，也能提问别的问题。

例10.各个铁路公司有多台车辆和多条路线。路线由多个车站构成。如果车站是同一个铁路公司的，车站可以被不同路线共用。路径由起点站和终点站构成。起点站和终点站可以在同一条路线上，也可以在不同的路线上。

<问题1>

为以下类图的a～e选择合适的选项，完成模型。



<选项>

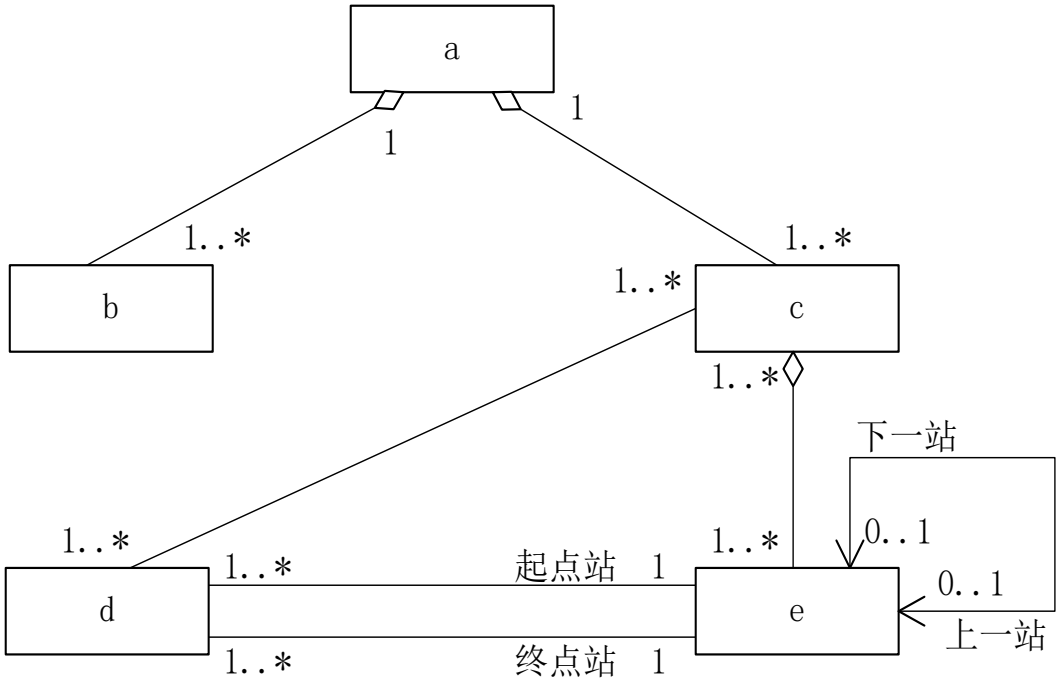
A．车站　　B．铁路公司　　C．路线

D．路径　　E．车辆

例11.各个铁路公司有多台车辆和多条路线。路线由多个车站构成。如果车站是同一个铁路公司的，车站可以被不同路线共用。路径由起点站和终点站构成。起点站和终点站可以在同一条路线上，也可以在不同的路线上。

<问题2>

请选择<u>不符合</u>问题1的类图的所有选项。



<选项>

A．路径的起点站和终点站有可能属于不同的铁路公司。

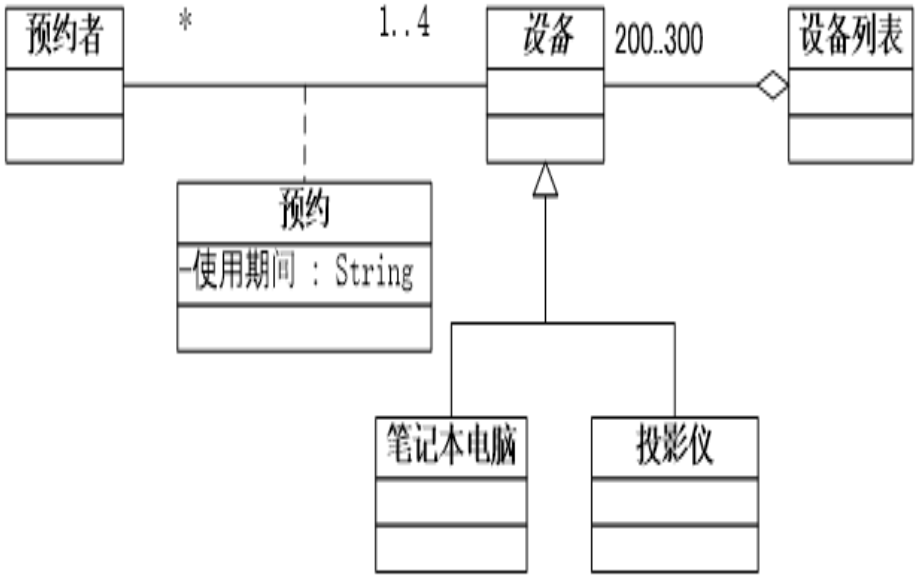
B．从某个车站到相邻的车站的多重性应该是[0..*]。

C．不同的铁路公司之间也可以共用路线。

D．通过该模型，不知道路线的第一个车站和最后一个车站。

例题参考答案

例题1：4
例题2：3
例题3：1， 4。
例题4： 2
例题5：



例题6：3
例题7：a-1 b-4 c-3 d-6 e-5 f-3 g-7 h-4 i-2
例题8：c
例题9：a b
例题10：a-B b-E c-C d-D e-A
例题11：C D

总结

- 类图表示系统内部的静态结构，对象图表示系统在某一时刻内部的对象状态，是系统的快照。
- 对象图可在类图设计时辅助决定类图中类之间关联的多重度关系。
- 类图中的类是由类名，属性和操作组成的。属性和操作区域可省略。还可对属性和操作指定可视性和作用域。
  - 属性区域和操作区域可省略，省略并不代表没有。
  - 可对属性和操作指定可视性（public,protected,private,package）
  - 可对属性和操作指定作用域（类作用域，实例作用域）
- 类之间的关系有
  - 关联（关联名，关联方向，关联端名，角色，多重度）
    - 可通过限定符降低关联的多重度
    - 拥有相关联的两个类之间关联信息的类，称为关联类。
    - 类之间不同意义的多个关系称为多重关联。
    - 同一个类的不同对象之间具有某种关系时，称为递归关联。
  - 泛化：超类和子类之间的继承关系。子类可继承超类的性质有属性，操作和关系。
    - 抽象类不能实例化，必须具有子类。
  - 聚合，组合：整体和部分的关系。
    - 组合是聚合的特殊形式，整体类的生命周期和部分类的生命周期一致。
  - 依赖—类之间使用上的关系。